Foundations of RL Learning: A Complete Theory and Experiment Guide

Author: GitHub tequila28

Project repository: https://github.com/tequila28/Foundations-of-RL-Learning

Motivation: this project starts from traditional reinforcement learning in order to build a solid foundation for understanding modern LLM reinforcement learning. Topics such as RLHF, RLVR, reward modeling, policy optimization, and reasoning-model training all rely on the same core ideas: policy, reward, value, sampling, optimization, exploration, and credit assignment.

The first eight chapters of this document are based on repository commit `ac68c8d041c23bc3985c4ea64fca02b940f3f419`; Chapter 9 draws on the project's new GSM8K, PPO, and GRPO code. This report connects nine chapters into one learning path: environment and value functions, planning with a known model, learning from sampled experience, function approximation and policy gradients, and finally LLM reinforcement learning.

You can read the project as a small reinforcement learning laboratory. The early chapters assume that the environment model is known, so transition matrices and Bellman equations can be written explicitly. The middle chapters remove that assumption and estimate values from sampled trajectories. The later chapters move beyond tabular representations and use features or neural networks to approximate values and policies.

0. Project Roadmap

Reinforcement learning studies how an agent makes sequential decisions through interaction with an environment. At each step, the agent observes a state `S_t`, selects an action `A_t`, receives a reward `R_{t+1}`, and moves to the next state `S_{t+1}`. The usual objective is to maximize discounted return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

where `gamma in [0,1)` is the discount factor. A larger `gamma` makes the agent care more about long-term rewards; a smaller `gamma` makes it focus more on immediate rewards.

The first eight chapters use a 5x5 GridWorld as their main setting. Chapter 9 transfers the same RL concepts to GSM8K math questions and token-by-token language-model generation:

Chapter	Topic	Core Question	Requires Environment Model
Chapter 1	Basic concepts and environment modeling	What are states, actions, rewards, policies, and values?	Yes
Chapter 2	Bellman equations	How do we evaluate a fixed policy?	Yes
Chapter 3	Dynamic programming	How do we find an optimal policy when the model is known?	Yes

Chapter	Topic	Core Question	Requires Environment Model
Chapter 4	Monte Carlo methods	How do we learn from complete sampled trajectories?	No
Chapter 5	Stochastic approximation	How do noisy samples drive iterative optimization?	No
Chapter 6	Temporal-difference learning	How do we learn online with bootstrapping?	No
Chapter 7	Value function approximation	How do we scale beyond tabular values?	No
Chapter 8	Policy gradient	How do we optimize a parameterized policy directly?	No
Chapter 9	LLM RL: PPO and GRPO	How do final-answer rewards optimize a token-generating policy?	No; uses sampled answers and rule rewards

One important implementation detail appears throughout the project: the target cell in GridWorld is not a terminal state. Reaching the target gives a positive reward, but the agent can continue acting. If it chooses `stay` at the target, it receives the positive reward repeatedly. Therefore the task behaves like a discounted continuing task rather than a maze that ends at the goal. With `gamma=0.9` and target reward `1`, the value of staying at the target is close to:

$$1 + 0.9 + 0.9^2 + \cdots = \frac{1}{1 - 0.9} = 10$$

This detail matters for every later plot. A high-value policy is not merely a policy that reaches the target once; it is a policy that can keep the agent near the rewarding state and continue collecting long-term reward.

0.1 Why Reinforcement Learning Needs an Environment

In supervised learning, the dataset is usually fixed: the model sees pairs `(x, y)` and learns a mapping from inputs to labels. Reinforcement learning adds another layer: the data distribution depends on the agent's own actions. The policy determines which states are visited, and the visited states determine which rewards and transitions the agent observes.

In GridWorld, a policy that always moves right will visit the right boundary often. A policy that actively seeks the target will visit the target neighborhood more frequently. The same algorithm can therefore learn different values under different exploration strategies. This is why later chapters repeatedly distinguish the behavior policy, the target policy, and the sampling distribution.

Core RL objects in this project can be mapped as follows:

Mathematical Object	Meaning	Project Representation
State `s`	The agent's current situation	One cell in the 5x5 grid

Mathematical Object	Meaning	Project Representation
Action `a`	A choice available to the agent	Up, right, down, left, stay
Reward `r`	Immediate feedback from the environment	Goal reward, boundary penalty, forbidden-cell penalty
Policy `pi(a|s)`	Action probabilities in each state	
Transition `P(s'|s,a)`	How actions move the agent	
Value `v(s)`	Long-term quality of a state	Numbers and colors in value plots

These objects are the shared language of the entire repository. DP, MC, TD, Q-learning, function approximation, and Actor-Critic all estimate or optimize some part of this structure.

1. Chapter 1: Basic Concepts and GridWorld Environment

1.1 Theory

A finite Markov decision process can be written as:

$$\text{MDP} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

where:

- `S` is the finite state set;
- `A` is the finite action set;
- `P(s'|s,a)` is the transition probability;
- `R(s,a)` is the immediate reward;
- `gamma` is the discount factor.

A policy `pi(a|s)` defines how the agent chooses actions in each state. Given a policy, the state-value function is:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

The action-value function is:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

They are related by:

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a)$$

$$q_\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) v_\pi(s')$$

The state value averages over the policy's action probabilities. The action value first fixes an action and then follows the policy afterward. This is why many control algorithms learn `Q(s,a)`: once action values are available, a greedy policy can be extracted without explicitly knowing the environment model.

Optimal values are defined as:

$$v^*(s) = \max_\pi v_\pi(s), \qquad q^*(s,a) = \max_\pi q_\pi(s,a)$$

An optimal policy can be selected by:

$$\pi^*(s) \in \arg\max_a q^*(s,a)$$

1.2 Implementation

Chapter 1 implements a 5x5 GridWorld. States are flattened using `row * 5 + col`, so the top-left cell is state 0 and the bottom-right cell is state 24. The action set is:

```
up, right, down, left, stay
```

Default configuration:

Item	Setting
Grid size	5x5
Number of states	25
Number of actions	5
Forbidden cells	6, 7, 12, 16, 18, 21
Target cell	17
Discount factor	0.9
Boundary penalty	-1
Forbidden-cell penalty	-1
Target reward	1
Default reward	0

Forbidden cells are not walls. The agent can enter them but receives a penalty. If an action moves the agent outside the grid, the agent stays in place and receives a boundary penalty.

1.2.1 How the Code Builds the GridWorld

The environment construction can be understood in four steps.

First, the two-dimensional grid is flattened into one-dimensional state indices:

```
state = row * size + col
row = state // size
col = state % size
```

This makes value functions simple vectors:

```
V = [V(0), V(1), ..., V(24)]
```

Second, each action is mapped to a coordinate change:

```
up    -> row - 1, col
right -> row, col + 1
down  -> row + 1, col
left  -> row, col - 1
stay  -> row, col
```

Third, for each `(state, action)` pair, the code fills in the next state and reward. Because this environment is deterministic, each state-action pair has exactly one successor:

$$P[s, a, \text{next_state}] = 1, \qquad P[s, a, \text{other}] = 0$$

$$R[s, a] = \text{reward}$$

The environment-building logic can be summarized as:

```
for state in all_states:
    row, col = decode(state)
    for action in all_actions:
        next_row, next_col = move(row, col, action)

        if out_of_bounds(next_row, next_col):
            next_state = state
            reward = r_bound
        else:
            next_state = encode(next_row, next_col)
            if next_state in forbidden:
                reward = r_forbid
            elif next_state in target:
                reward = r_target
            else:
                reward = r_default

        P[state, action, next_state] = 1
        R[state, action] = reward
```

Fourth, a policy is represented as a matrix:

```
policy.shape = (num_states, num_actions)
```

The policy-induced transition matrix and reward vector are:

$$P_\pi[s, s'] = \sum_a \pi(a \mid s) P[s, a, s']$$

$$r_\pi[s] = \sum_a \pi(a \mid s) R[s, a]$$

This turns the MDP into a Markov reward process under a fixed policy, which is exactly what Chapter 2 evaluates.

The nonterminal target detail is also visible in the model:

$$R(17, \text{stay}) = 1, \qquad P(17 \mid 17, \text{stay}) = 1$$

Therefore:

$$V(17) = 1 + 0.9V(17) = 10$$

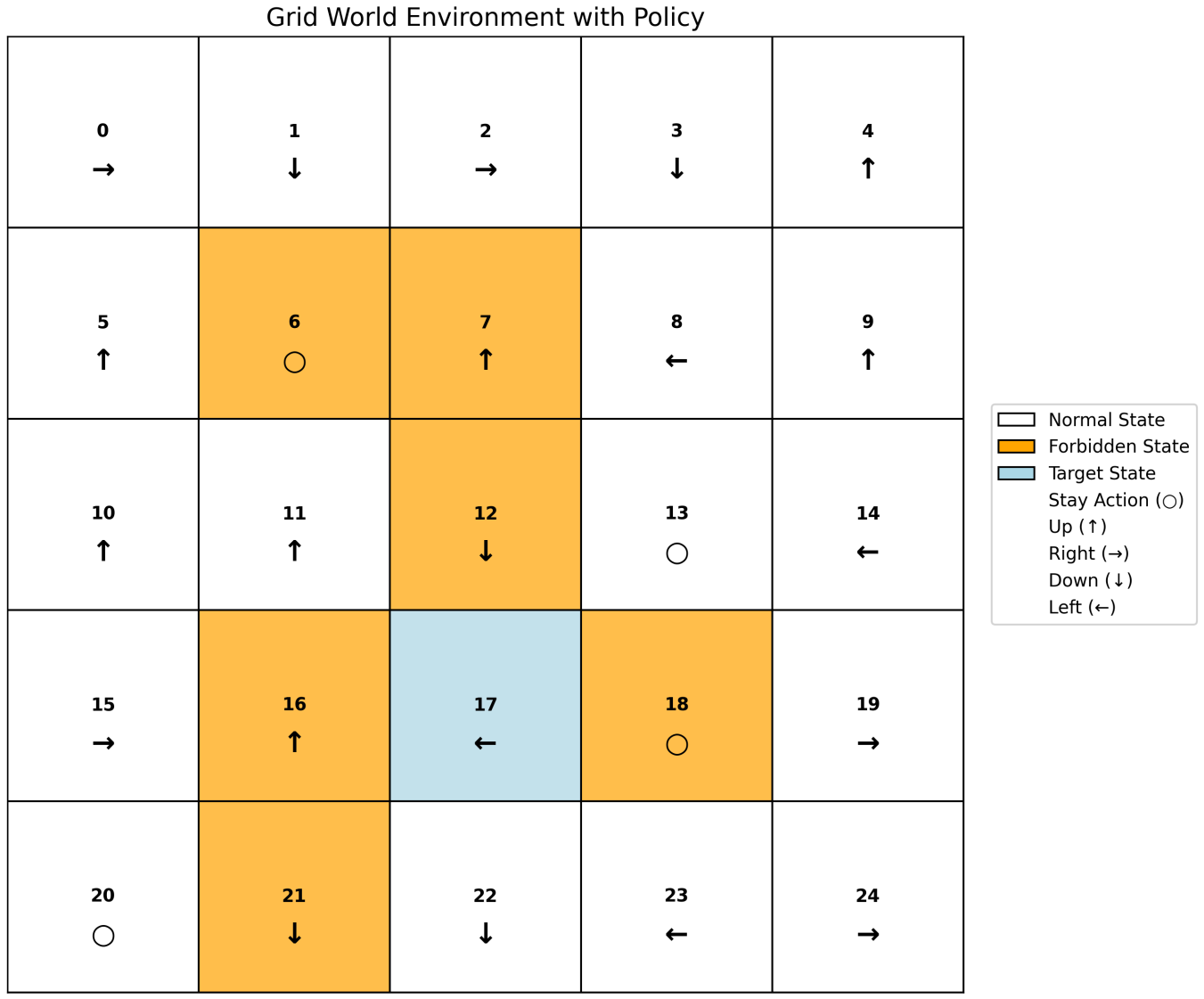
1.2.2 What This Chapter Does and Why

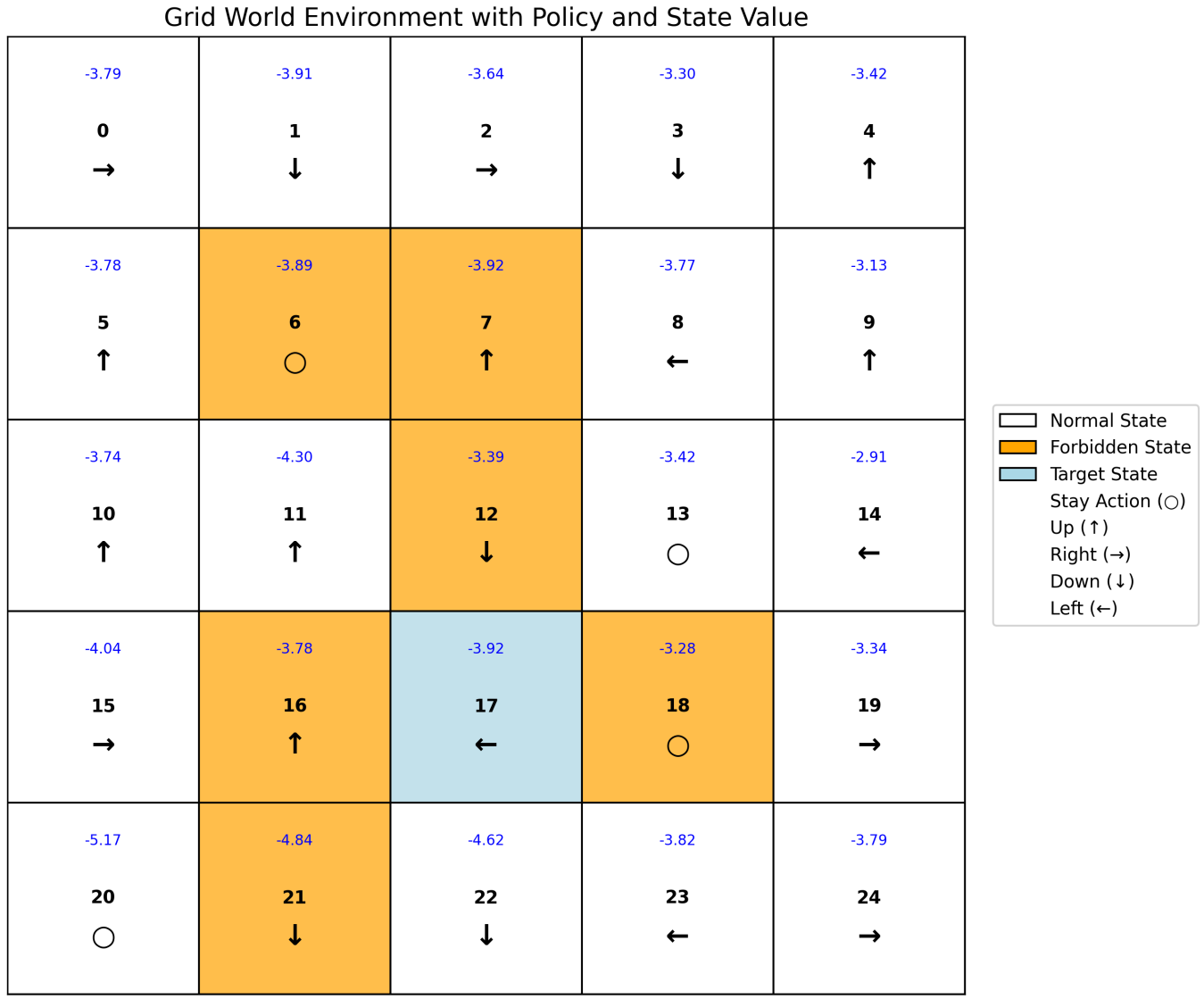
Chapter 1 does not train an agent. It builds the world shared by the rest of the project. It defines the state space, action space, transition model, and reward rules. Without this step, Bellman equations would have no `P` or `R`, dynamic programming would have no model to solve, and MC/TD methods would have no environment to sample from.

This chapter also turns a policy from an intuitive rule into a concrete probability table. A policy is not merely "move toward the goal"; in RL, it is a distribution over actions in every state. Once represented as a matrix, it can be evaluated, sampled, visualized, and improved.

Step	What It Does	Why It Matters
Build GridWorld	Creates 25 states, 5 actions, rewards, and transitions	Provides a shared benchmark
Build a random policy	Assigns action probabilities in every state	Shows policies as probability matrices
Evaluate the policy	Computes values under the policy	Turns long-term return into numbers
Visualize policy and value	Plots arrows and state values	Makes model behavior easy to inspect

1.3 Experimental Results


Grid World Environment with Policy
0 →
1 ↓
2 →
3 ↓
4 ↑
5 ↑
6 ○
7 ↑
8 ←
9 ↑
10 ↑
11 ↑
12 ↓
13 ○
14 ←
15 →
16 ↑
17 ←
18 ○
19 →
20 ○
21 ↓
22 ↓
23 ←
24 →
Normal State
Forbidden State
Target State
Stay Action (○)
Up (↑)
Right (→)
Down (↓)
Left (←)



The first plot shows action probabilities in each state. The second plot shows the corresponding value function. Because the policy is random, the agent often hits boundaries, enters forbidden cells, or leaves the target cell. The value landscape is therefore much less focused than it would be under an optimal policy.

Chapter 1 matters because it grounds abstract RL concepts in concrete arrays and plots. All later Bellman, DP, MC, and TD updates reuse this same environment structure.

2. Chapter 2: Bellman Equations

2.1 Theory

The Bellman equation is the central recursive identity of reinforcement learning. Given a fixed policy `pi`, the state value satisfies:

$$v_\pi(s) = r_\pi(s) + \gamma \sum_{s'} P_\pi(s' \mid s) v_\pi(s')$$

where:

$$r_\pi(s) = \sum_a \pi(a \mid s) R(s, a)$$

$$P_\pi(s' \mid s) = \sum_a \pi(a \mid s) P(s' \mid s, a)$$

In matrix form:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

The closed-form solution is:

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$$

In practice, solving the linear system is usually preferred:

$$(I - \gamma P_\pi) v_\pi = r_\pi$$

An iterative alternative is:

$$v_{k+1} = r_\pi + \gamma P_\pi v_k$$

Two Bellman equations should be distinguished. The expectation equation evaluates a fixed policy:

$$v_\pi(s) = \sum_a \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s, a) v_\pi(s') \right]$$

The optimality equation takes the best action:

$$v^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s, a) v^*(s') \right]$$

Chapter 2 uses the expectation equation because it evaluates fixed policies. Chapter 3 moves to the optimality equation because it searches for optimal policies.

2.2 Implementation

Chapter 2 compares two policy evaluation methods:

- Closed-form evaluation through a linear system;
- Iterative Bellman evaluation until the maximum value change is below a tolerance.

The experiment evaluates several policies, including hand-designed policies, a poor always-right-style policy, and a randomly normalized policy. The last one is random but not exactly uniform over actions.

2.2.1 What This Chapter Implements and Why

Chapter 2 answers a specific question: given a policy, how good is it? Chapter 1 provides the environment and policy representation. Chapter 2 combines them into `P_pi` and `r_pi`, then computes the long-term value of every state.

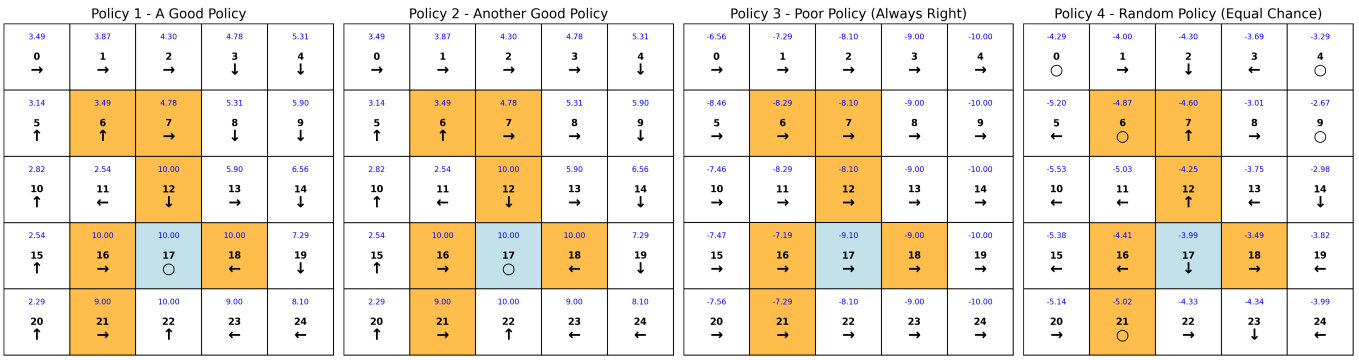
Module	What It Does	Related Theory
Policy-induced model	Averages `P[s,a,s']` and `R[s,a]` under `pi(a | s)`	
Closed-form solver	Solves `(I-\gamma P_\pi)v=r_\pi`	Matrix Bellman equation

Module	What It Does	Related Theory
Iterative solver	Repeatedly applies `v <- r + gamma Pv`	Bellman contraction
Policy comparison	Visualizes value functions under different policies	Policy determines return

The closed-form solution shows that policy value is a linear equation problem in small finite MDPs. The iterative solution shows how value can propagate through repeated Bellman backups, which is closer to DP and TD learning. This chapter prepares the key idea used later: once a policy can be evaluated, it can also be improved.

2.3 Experimental Results





The two plots show that closed-form and iterative evaluation produce nearly the same value functions. In an additional numerical check, the maximum difference under the same uniform policy was about `9.2e-6`, showing that the iterative method is very close to the linear-system solution under the chosen tolerance.

3. Chapter 3: Policy Iteration, Value Iteration, and Truncated Policy Iteration

3.1 Theory

Dynamic programming assumes that `P` and `R` are known. Policy iteration alternates between evaluation and improvement:

$$\text{Policy Evaluation:} \quad v_\pi \leftarrow r_\pi + \gamma P_\pi v_\pi$$

$$\text{Policy Improvement:} \quad \pi_{\text{new}}(s) \leftarrow \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v_\pi(s') \right]$$

The policy improvement theorem states that if:

$$q_\pi(s, \pi_{\text{new}}(s)) \geq v_\pi(s)$$

then the new policy is no worse than the old one.

Value iteration directly applies the Bellman optimality backup:

$$v_{k+1}(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v_k(s') \right]$$

After convergence, the greedy policy is:

$$\pi^*(s) = \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v^*(s') \right]$$

Truncated policy iteration evaluates the current policy only for a fixed number of sweeps before improving it. It sits between full policy iteration and value iteration, showing the tradeoff between evaluation accuracy and computation cost.

3.2 Implementation

Algorithm	Project Behavior
Policy Iteration	Iteratively evaluates the current policy, then greedily improves it
Value Iteration	Performs optimal Bellman updates and extracts a greedy policy
Truncated Policy Iteration	Runs only x evaluation sweeps before policy improvement

The implementation uses in-place state updates during value iteration and policy evaluation. In this small deterministic environment, this does not change the conceptual algorithm, but it can affect iteration counts and convergence curves.

3.2.1 What This Chapter Implements and Why

Chapter 3 moves from policy evaluation to control. The agent no longer only asks "how good is this policy?" It asks "how can I systematically find a better policy?"

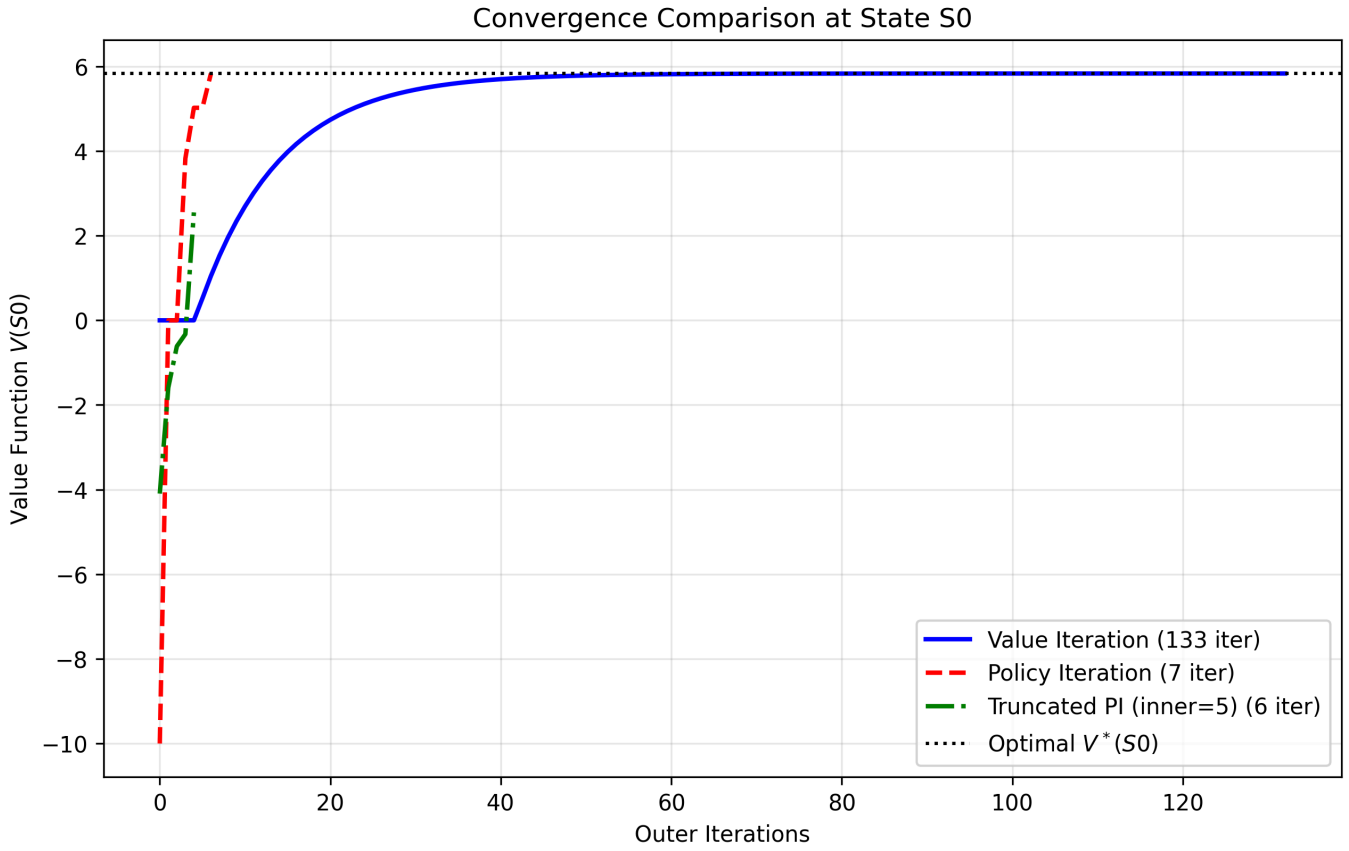
Method	What It Does	Why It Is Included
Policy Iteration	Fully evaluates the current policy, then improves it	Shows the clean evaluation-improvement loop
Value Iteration	Updates values directly toward the optimal Bellman target	Shows that intermediate policies need not be fully evaluated
Truncated PI	Partially evaluates before improving	Shows the cost-accuracy tradeoff

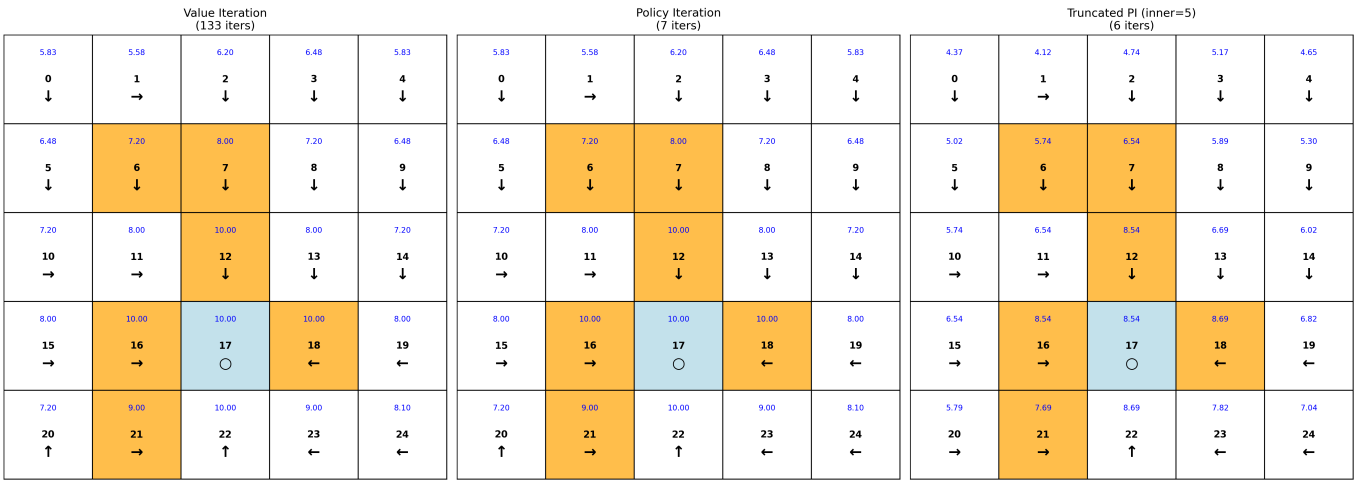
This chapter is placed before MC and TD because DP is the cleanest case: the model is known, there is no sampling noise, and value propagation is fully controlled by Bellman equations. Later model-free methods approximate this process without access to the full model.

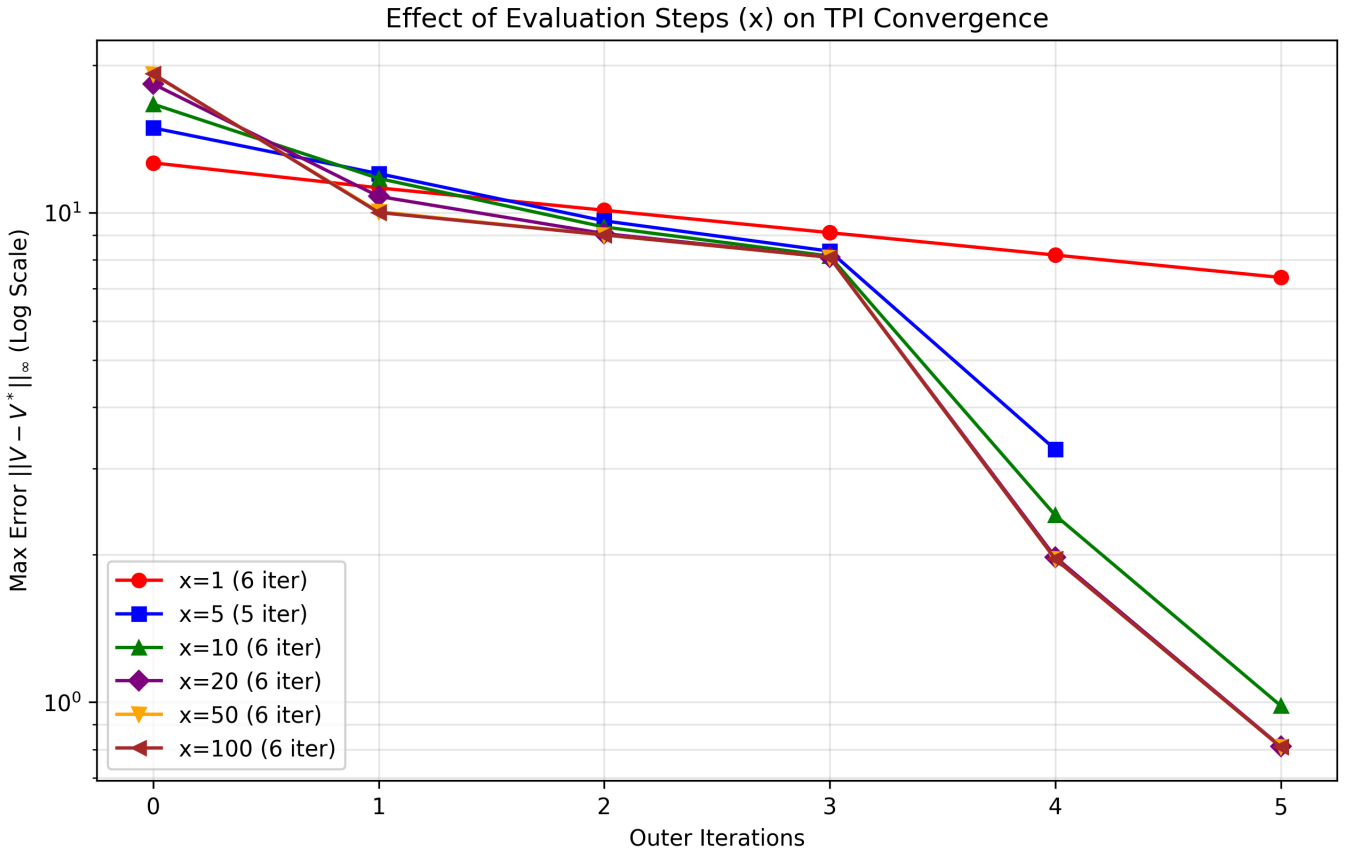
Additional checks:

Check	Result
Max value difference between VI and PI	`6.84e-6`
Target `stay` next state	17
Target `stay` immediate reward	`1.0`
Optimal target value with `gamma=0.9`	about `9.99999`

3.3 Experimental Results







The convergence plot compares how quickly different algorithms improve the value of state 0. TPI demonstrates the effect of x: with too few evaluation sweeps, value estimates remain rough; with larger x, TPI behaves closer to full policy iteration.

Additional final TPI value errors:

x	Final Value Error
1	6.6309
5	1.4596
10	0.0605
20	0.00028
50	8.21e-6
100	8.21e-6

4. Chapter 4: Monte Carlo Methods

4.1 Theory

Dynamic programming requires the model. Monte Carlo methods learn from sampled trajectories. For a state-action pair:

$$Q(s,a) \leftarrow \text{Average}(G_t \mid S_t = s, A_t = a)$$

For an episode:

```
S_0, A_0, R_1, S_1, A_1, R_2, ...
```

the return from time `t` is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$

MC does not need a model and does not bootstrap. It waits for sampled returns. This makes it simple and unbiased for episodic tasks under suitable assumptions, but often high-variance and slow.

Exploring starts randomly chooses initial `(s,a)` pairs to improve coverage. Epsilon-greedy control keeps a small probability of exploration while mostly selecting the current greedy action.

4.2 Implementation

Method	Sampling Strategy	Update Behavior
Basic MC	Traverses state-action pairs and generates short trajectories	Length-10 trajectories
Exploring Starts	Random initial state-action pairs	10000 sampled trajectories
Epsilon-Greedy MC	Continual epsilon-greedy exploration	Every-visit style updates

For Basic MC and Exploring Starts, the code traverses trajectories backward and removes repeated visits. This selects the last visit in forward time rather than the standard first visit. The implementation still estimates returns from experience, but it should not be described as textbook first-visit MC.

Because this GridWorld is continuing, finite-length MC trajectories introduce truncation bias. With `gamma=0.9`, reward bound 1, and trajectory length 10:

$$\frac{\gamma^{10}}{1-\gamma} = \frac{0.9^{10}}{0.1} \approx 3.49$$

4.2.1 What This Chapter Implements and Why

Chapter 4 starts model-free learning. Instead of using the known transition matrix, the algorithms estimate values from trial experience.

Experiment	What It Does	Why It Is Included
Basic MC	Generates fixed-length trajectories from many state-action pairs	Ensures broad state-action estimation
Exploring Starts	Samples random initial `(s,a)` pairs	Improves exploration coverage
Epsilon-Greedy MC	Explores within trajectories	More closely resembles practical control

Exploration matters because the agent cannot evaluate actions it never tries. MC control is not difficult because of its formula; it is difficult because good estimates require sufficient coverage and enough trajectories.

4.3 Experimental Results

MC Basic
Final Policy

2.00 0 ↓	1.74 1 →	2.17 2 ↓	2.52 3 ↓	2.03 4 ↓
2.44 5 ↓	3.30 6 ↓	3.75 7 ↓	2.94 8 ↓	2.53 9 ←
2.98 10 →	3.66 11 →	4.95 12 ↓	3.51 13 ↓	3.20 14 ←
3.69 15 →	5.20 16 →	1.00 17 ○	5.13 18 ←	3.88 19 ←
3.38 20 ↑	5.12 21 →	5.45 22 ↑	4.95 23 ←	4.23 24 ←

MC Exploring Starts
Final Policy

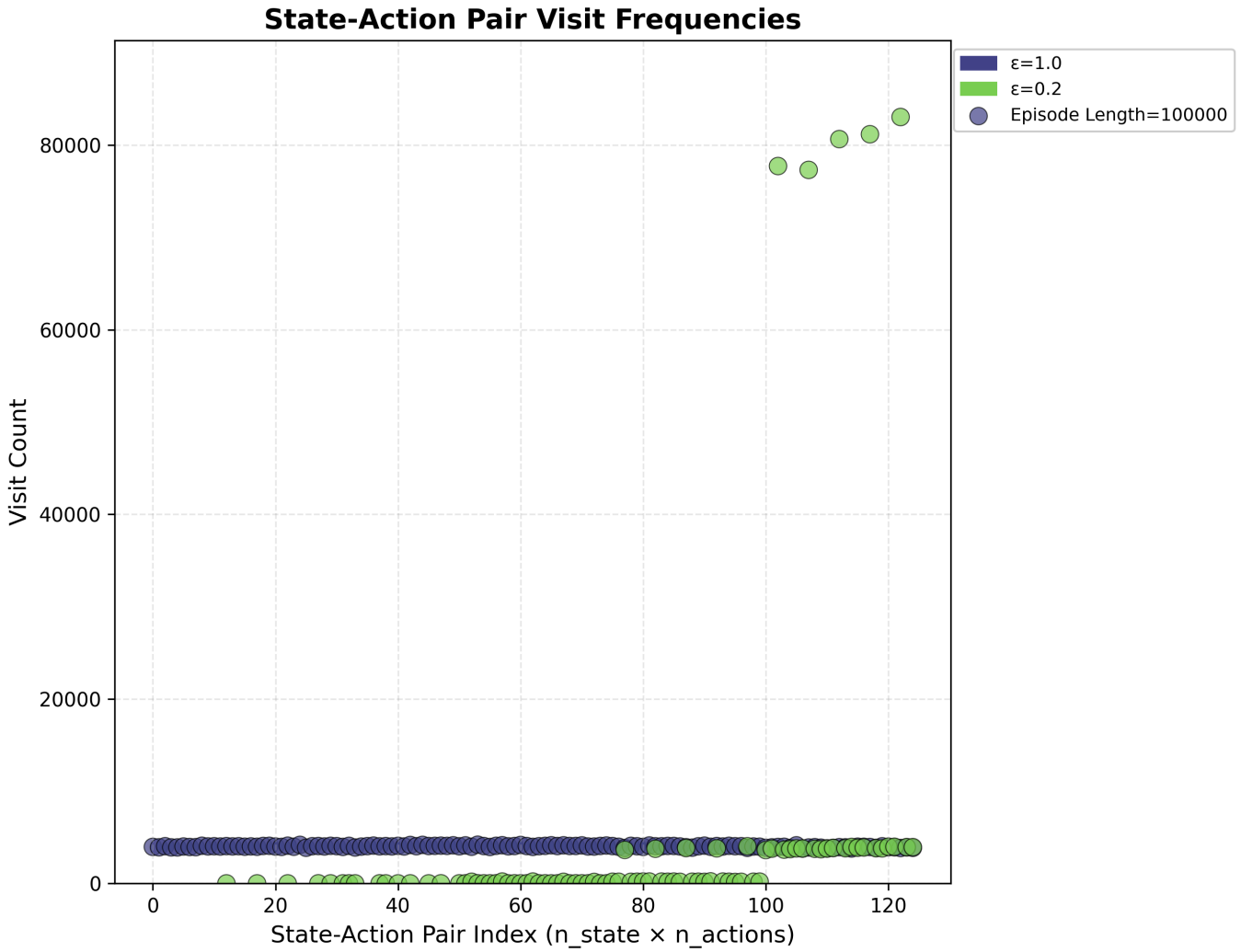
1.75 0 ↓	1.59 1 →	2.04 2 ↓	2.25 3 ↓	1.80 4 ←
2.16 5 ↓	3.03 6 ↓	3.58 7 ↓	2.71 8 ↓	2.45 9 ←
2.71 10 ↓	3.58 11 →	4.88 12 ↓	3.26 13 ↓	3.17 14 ←
3.29 15 →	4.79 16 →	1.00 17 ○	4.88 18 ←	3.57 19 ←
3.06 20 ↑	3.91 21 ↑	3.17 22 →	3.70 23 ↑	3.25 24 ←

MC ε-greedy (ε=0.1)
Final Policy

-0.33 0 →	3.87 1 ↓	2.36 2 ←	1.51 3 ←	-0.18 4 ←
3.04 5 ↓	5.74 6 ↓	0.21 7 →	1.62 8 ↑	-0.02 9 ○
4.86 10 →	6.47 11 →	8.56 12 ↓	0.73 13 ↑	0.78 14 ←
3.56 15 ↑	8.48 16 →	8.56 17 ○	8.26 18 ←	0.00 19 ↑
3.99 20 ↑	7.46 21 →	8.00 22 ↑	4.38 23 ←	-1.78 24 ←

MC ε-greedy (ε=0.2)
Final Policy

0.48 0 ↓	1.43 1 ○	0.88 2 →	0.19 3 ↓	-0.32 4 ↓
1.03 5 ↓	0.54 6 ←	4.56 7 ↓	2.65 8 ←	0.21 9 ←
4.30 10 ↓	3.55 11 ←	7.13 12 ↓	4.68 13 ←	-0.11 14 ↑
4.96 15 →	7.16 16 →	7.22 17 ○	7.17 18 ←	0.82 19 ↑
1.56 20 ↑	5.71 21 →	7.18 22 ↑	5.64 23 ←	3.56 24 ←



The policy comparison shows how different sampling mechanisms affect learned greedy policies and values. The scatter plot visualizes state-action coverage. Sparse visits lead to unstable estimates, which is exactly why exploration assumptions matter in MC theory.

5. Chapter 5: Stochastic Approximation and Optimization

5.1 Theory

Stochastic approximation explains how noisy samples can drive iterative learning. Suppose the target is:

$$\theta^* = \mathbb{E}[X]$$

A natural update is:

$$\theta_{k+1} = \theta_k + \alpha_k(X_k - \theta_k)$$

For the objective:

$$J(\theta) = \frac{1}{2}\mathbb{E}\left[\|\theta - X\|^2\right]$$

the gradient is:

$$\nabla J(\theta) = \theta - \mathbb{E}[X]$$

Using one sample gives the stochastic update:

$$\theta_{k+1} = \theta_k - \alpha_k(\theta_k - X_k)$$

Classical stochastic approximation often requires:

$$\sum_k \alpha_k = \infty, \qquad \sum_k \alpha_k^2 < \infty$$

5.2 Implementation

Chapter 5 uses a two-dimensional mean estimation problem. The environment samples 1000 points from `[-25,25]^2`, with true mean `[0,0]`. Algorithms start from `[50,50]` and run 400 updates.

Method	Update
BGD	Uses the full sample mean
SGD	Uses one random sample
MBGD	Uses a mini-batch mean

The recorded error is distance to the true mean `[0,0]`, not necessarily distance to the finite-sample mean. Therefore the curve includes both optimization error and finite-sample statistical error.

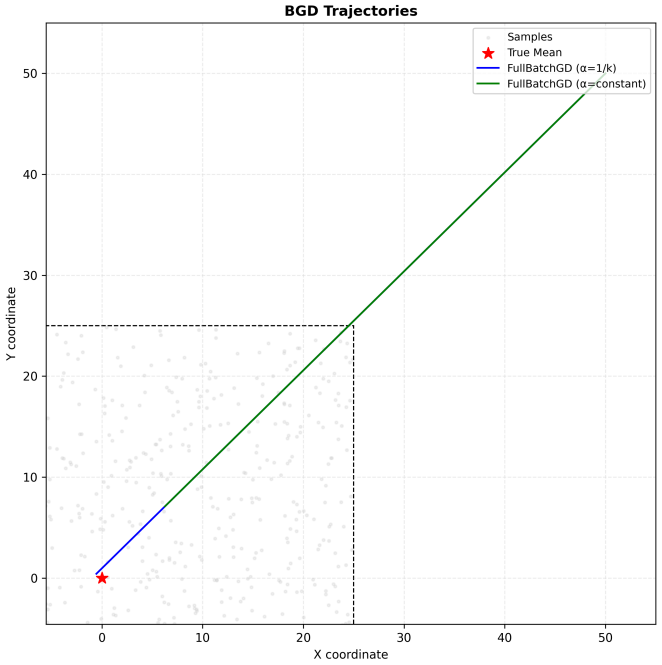
5.2.1 What This Chapter Implements and Why

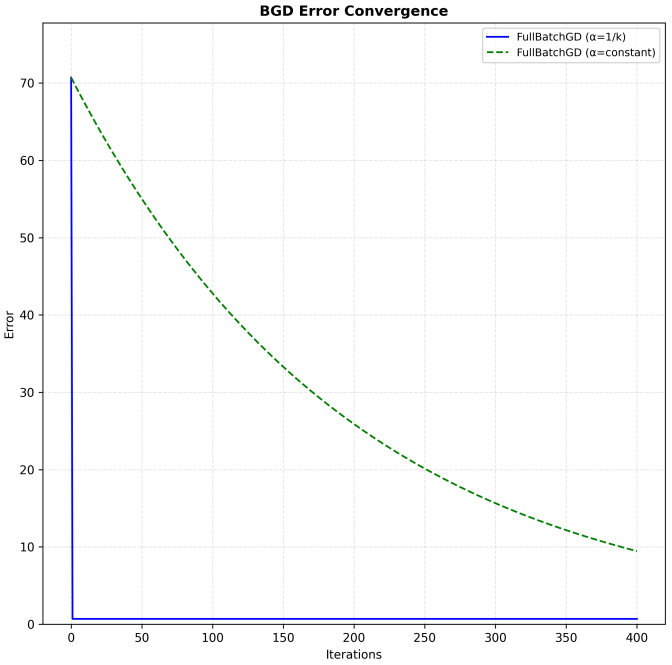
This chapter temporarily leaves GridWorld, but it builds the optimization intuition needed for RL. MC, TD, Q-learning, and policy gradient all update estimates using noisy samples.

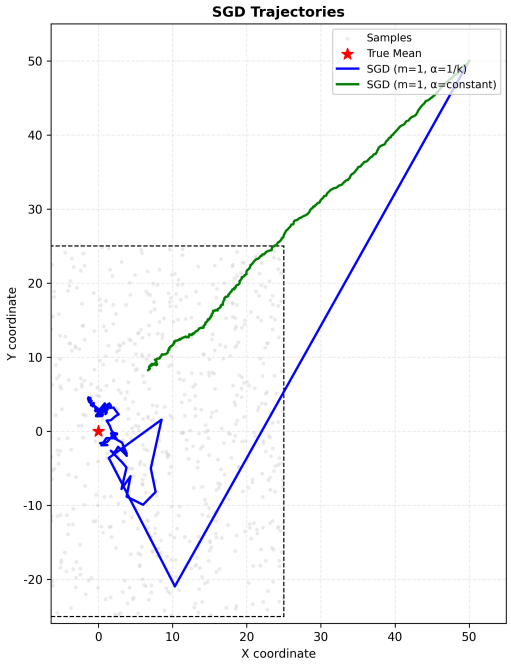
Method	What It Does	Why It Matters
BGD	Updates using all samples	Low-noise but more expensive
SGD	Updates using one sample	High-noise but online and cheap
MBGD	Updates using a small batch	Balances noise and computation

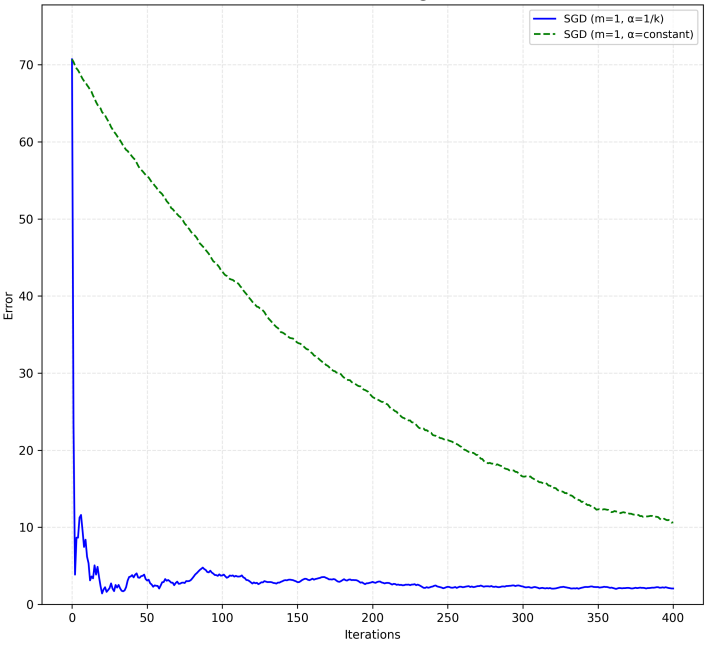
TD learning behaves much like SGD: each sampled transition creates a noisy target, and the algorithm moves the current estimate toward that target.

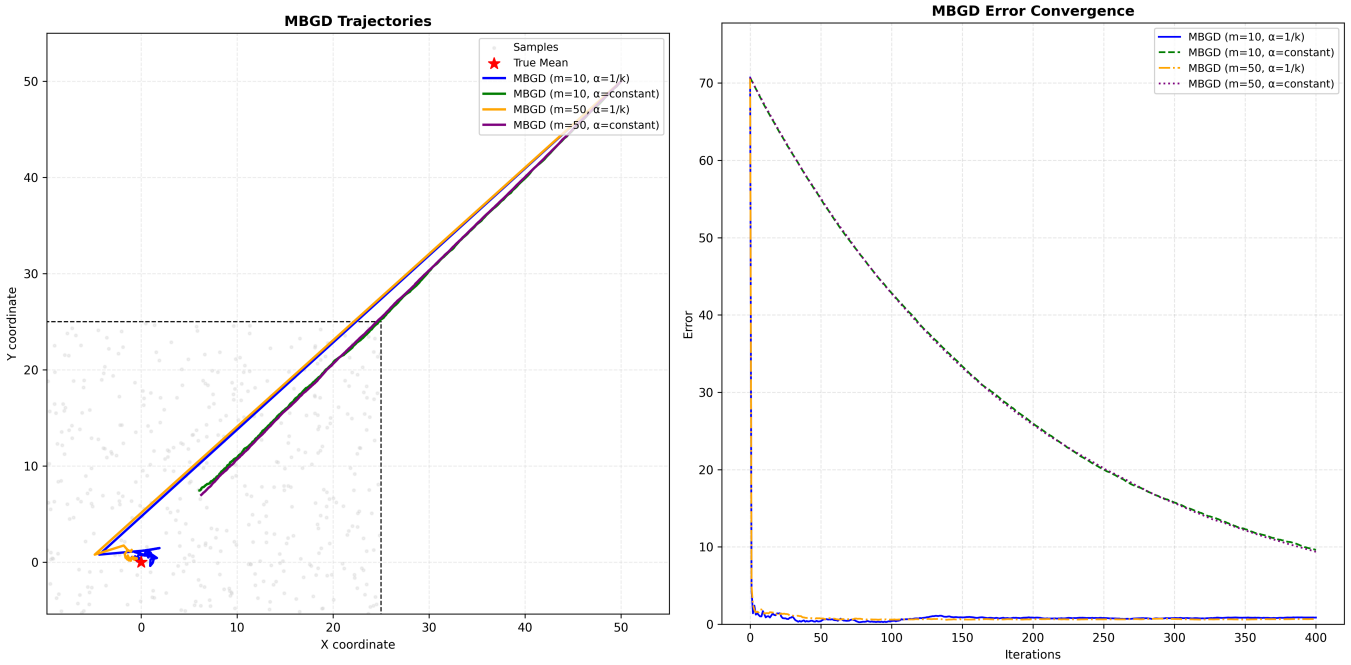
5.3 Experimental Results


BGD Trajectories
Samples
True Mean
FullBatchGD (α=1/k)
FullBatchGD (α=constant)
Y coordinate
X coordinate


BGD Error Convergence
FullBatchGD (α=1/k)
FullBatchGD (α=constant)
Error
Iterations


SGD Trajectories
Samples
True Mean
SGD (m=1, α=1/k)
SGD (m=1, α=constant)
Y coordinate
X coordinate


SGD Error Convergence
SGD (m=1, α=1/k)
SGD (m=1, α=constant)
Error
Iterations



BGD is smooth because it uses all samples. SGD is noisy because every update depends on a single point. MBGD sits between the two. This chapter helps explain why RL reward and TD-error curves often fluctuate.

6. Chapter 6: Temporal-Difference Learning

6.1 Theory

TD learning combines MC and DP:

- Like MC, it learns from sampled experience without the model;
- Like DP, it bootstraps from current value estimates.

TD(0) updates state value by:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]$$

The bracketed term is the TD error:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

SARSA:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$$

Q-learning:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right]$$

Expected SARSA:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\right]$$

n-step SARSA interpolates between one-step TD and Monte Carlo by using multiple rewards before bootstrapping.

The key distinction is the target:

$$\text{target}_{MC} = G_t$$

$$\text{target}_{TD} = R_{t+1} + \gamma V(S_{t+1})$$

6.2 Implementation

Algorithm	Project Goal
SARSA	On-policy epsilon-greedy TD control
Expected SARSA	Expected next value under epsilon-greedy policy
n-step SARSA	Comparison of `n=1` and `n=10`
Online Q-learning	Step-by-step Q-learning updates
Offline-style Q-learning	Collects a trajectory first, then updates in order

`Online Q-learning` is named with an on-policy flavor in the code, but the update target uses `max_a Q(s',a)`, so it is conceptually Q-learning. The offline-style version refers to the update workflow, not modern fixed-dataset offline RL.

Episodes run for 1000 steps and do not terminate upon reaching the target. Epsilon starts at `0.2`, decays by `0.998`, and bottoms out at `0.01`.

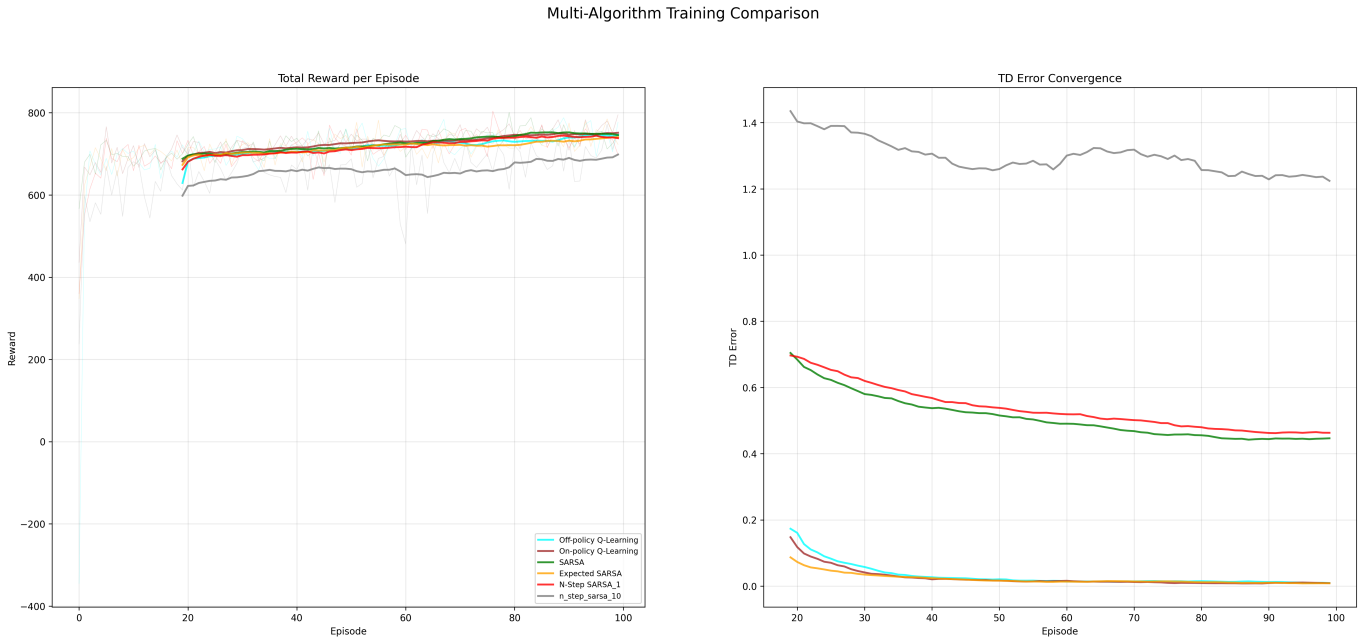
6.2.1 What This Chapter Implements and Why

Chapter 6 is the core tabular model-free control chapter. MC waits for complete trajectories; TD can update after every step. This makes TD more online and more sample-efficient in many cases.

Layer	Implementation	Purpose
One-step TD control	SARSA, Expected SARSA, Q-learning	Compare different TD targets
Multi-step TD control	n-step SARSA	Study the MC-TD tradeoff
Update workflow	Online vs collected-trajectory Q-learning	Compare interaction/update timing

SARSA uses the actually sampled next action, Expected SARSA averages over the next policy, and Q-learning uses the greedy next action. This chapter prepares the structure later reused by value approximation and critics in actor-critic methods.

6.3 Experimental Results



Final Policies Learned by Different TD Algorithms

Off-policy Q-Learning
ε=0.1637

1.14 0 ↓	0.12 1 ←	0.53 2 →	2.44 3 ↓	1.84 4 ↓
5.13 5 ↓	2.36 6 ←	8.00 7 ↓	4.74 8 →	5.85 9 ↓
7.07 10 →	8.00 11 ↓	10.00 12 ↓	5.90 13 →	6.56 14 ↓
8.00 15 →	10.00 16 →	10.00 17 ○	10.00 18 ←	7.29 19 ↓
5.05 20 ↑	9.00 21 →	10.00 22 ↑	9.00 23 ←	8.10 24 ←

On-policy Q-Learning
ε=0.1637

1.81 0 ↓	0.43 1 ←	0.02 2 ←	1.03 3 ↓	0.35 4 ↓
4.47 5 ↓	6.09 6 ↓	8.00 7 ↓	6.11 8 ↓	3.34 9 ↓
7.00 10 →	8.00 11 →	10.00 12 ↓	8.00 13 ←	6.52 14 ↓
8.00 15 →	10.00 16 →	10.00 17 ○	10.00 18 ←	8.00 19 ←
5.29 20 →	9.00 21 →	10.00 22 ↑	9.00 23 ←	6.75 24 ←

SARSA
ε=0.1637

1.05 0 ↓	0.17 1 ←	0.50 2 →	2.58 3 ↓	0.19 4 ←
2.59 5 ↓	3.43 6 ↓	4.07 7 →	4.71 8 ↓	1.61 9 ←
4.65 10 →	5.68 11 →	7.28 12 ↓	5.12 13 ↓	3.76 14 ←
5.57 15 →	7.46 16 →	7.78 17 ○	7.41 18 ←	5.61 19 ←
4.07 20 →	6.66 21 →	7.62 22 ↑	6.51 23 ←	4.69 24 ←

Expected SARSA
ε=0.1637

0.71 0 ↓	0.13 1 →	0.67 2 →	2.54 3 ↓	0.17 4 ←
2.92 5 ↓	3.92 6 ↓	3.63 7 →	4.63 8 ↓	1.95 9 ←
4.64 10 →	5.54 11 →	7.76 12 ↓	5.54 13 ←	4.01 14 ←
3.70 15 ↑	7.75 16 →	7.76 17 ○	7.75 18 ←	4.36 19 ↓
2.20 20 ↑	5.52 21 ↑	7.75 22 ↑	6.67 23 ←	5.54 24 ←

N-step SARSA (n=1)
ε=0.1637

1.17 0 ↓	0.35 1 ←	-0.02 2 →	0.57 3 ↓	0.21 4 ↓
2.60 5 ↓	4.00 6 ↓	5.37 7 ↓	3.20 8 ↓	1.38 9 ↓
4.10 10 →	4.91 11 ↓	7.25 12 ↓	5.02 13 ←	4.19 14 ←
5.24 15 →	7.16 16 →	6.96 17 ○	7.36 18 ←	4.96 19 ←
3.04 20 →	4.97 21 ↑	7.23 22 ↑	6.42 23 ←	4.72 24 ←

N-step SARSA (n=10)
ε=0.1637

1.12 0 ↓	0.52 1 →	1.42 2 →	2.10 3 ↓	1.22 4 ↓
1.92 5 ↓	1.06 6 →	2.64 7 →	3.26 8 ↓	2.28 9 ←
3.00 10 →	4.49 11 ↓	7.38 12 ↓	4.64 13 ←	2.79 14 ←
4.22 15 →	7.12 16 →	6.43 17 ↓	7.04 18 ←	4.50 19 ←
2.75 20 ↑	4.27 21 →	7.37 22 ↑	4.18 23 ←	2.97 24 ↑

The curves show episode rewards and TD errors. Q-learning and Expected SARSA show faster TD-error reduction in this run, while SARSA and n-step methods remain higher. TD error should not be read as a direct ranking of policy quality because algorithms use different targets and sampling paths.

7. Chapter 7: Value Function Approximation

7.1 Theory

Tabular methods store a separate value for every state or state-action pair. This becomes impossible when the state space is large or continuous. Function approximation uses parameterized functions:

$$\hat{V}(s;w) \approx V(s), \qquad \hat{Q}(s,a;w) \approx Q(s,a)$$

For linear value approximation:

$$\hat{V}(s;w) = w^\top \phi(s)$$

TD(0) becomes:

$$\delta_t = R_{t+1} + \gamma w^\top \phi(S_{t+1}) - w^\top \phi(S_t)$$

$$w \leftarrow w + \alpha \delta_t \phi(S_t)$$

For action values:

$$\hat{Q}(s,a;w_a) = w_a^\top \phi(s)$$

Function approximation allows generalization across states, but also introduces approximation error and stability concerns.

7.1.1 What This Chapter Implements and Why

Chapter 7 answers a practical question: what if we cannot store a table entry for every state? The project uses polynomial features and linear models because they are more general than tables but still interpretable.

Experiment	Implementation	Why It Is Included
Experiment 1	Linear TD(0) for state-value prediction	Compares feature orders
Experiment 2	Linear SARSA and Q-learning control	Tests control under approximation

Prediction learns values under a sampling process. Control is harder because value estimates influence action choices, which then change the future data distribution.

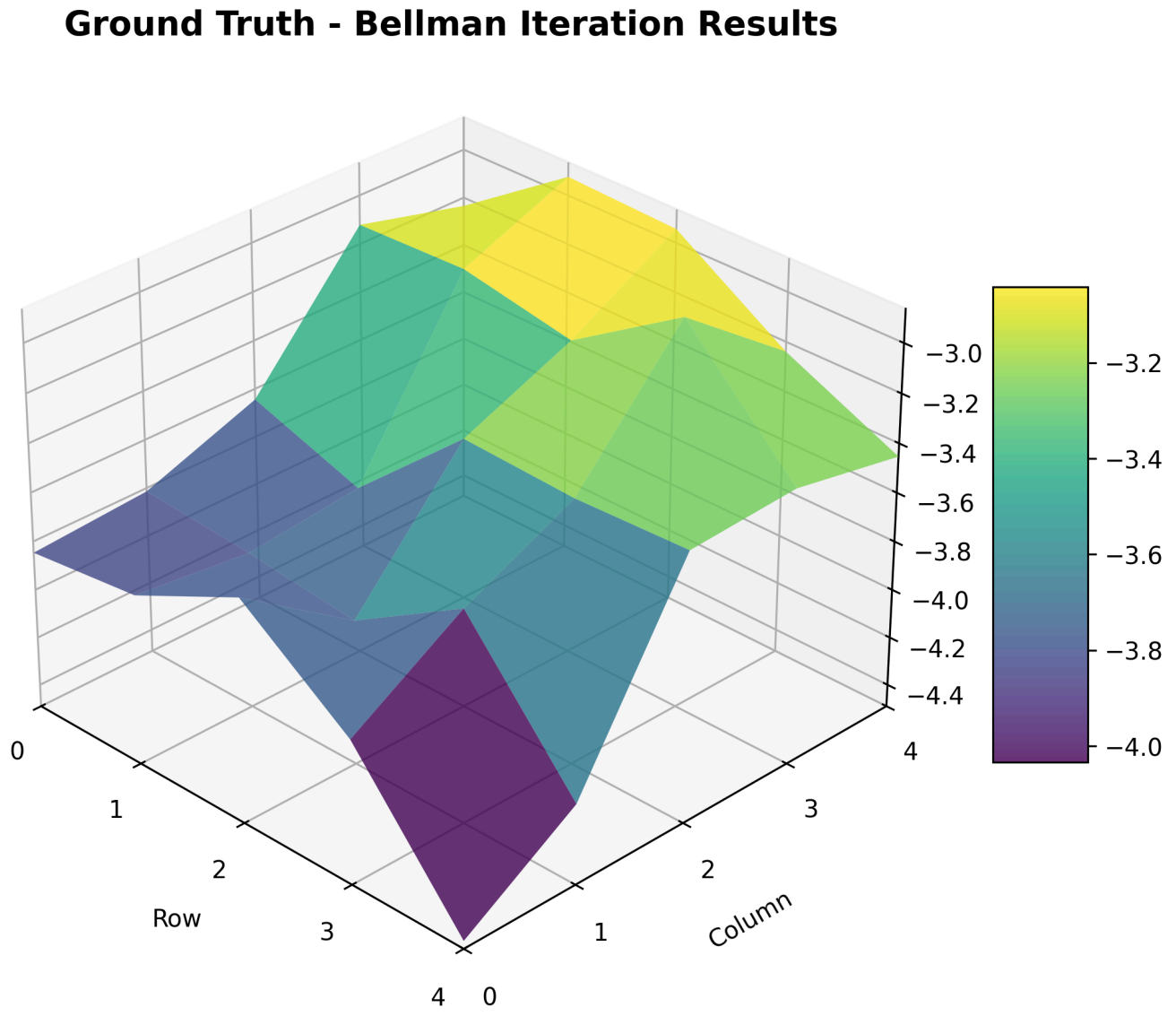
7.2 Experiment 1: Linear TD Prediction

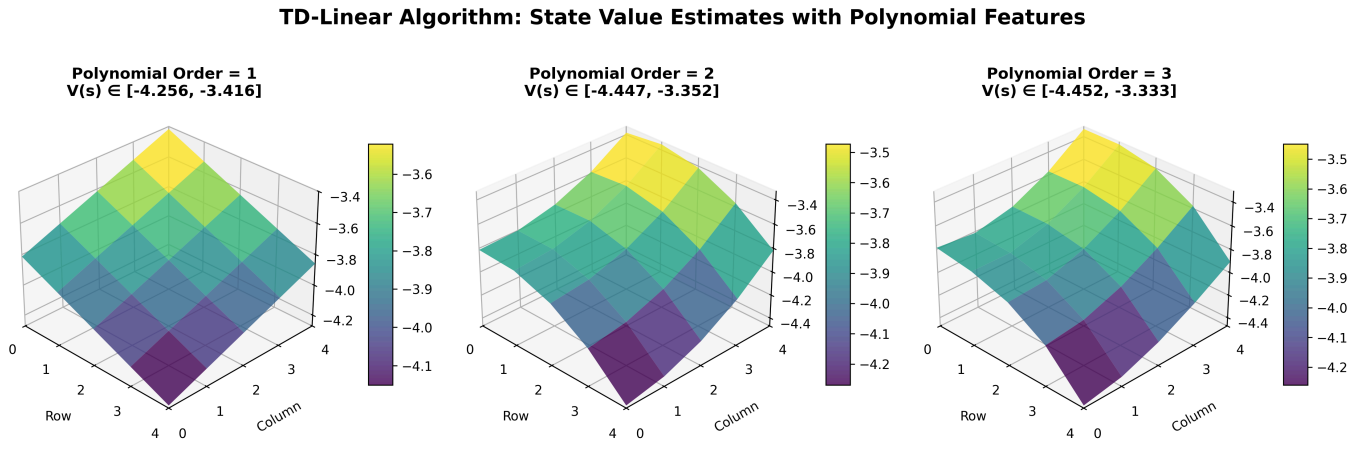
State coordinates are normalized and expanded into polynomial features:

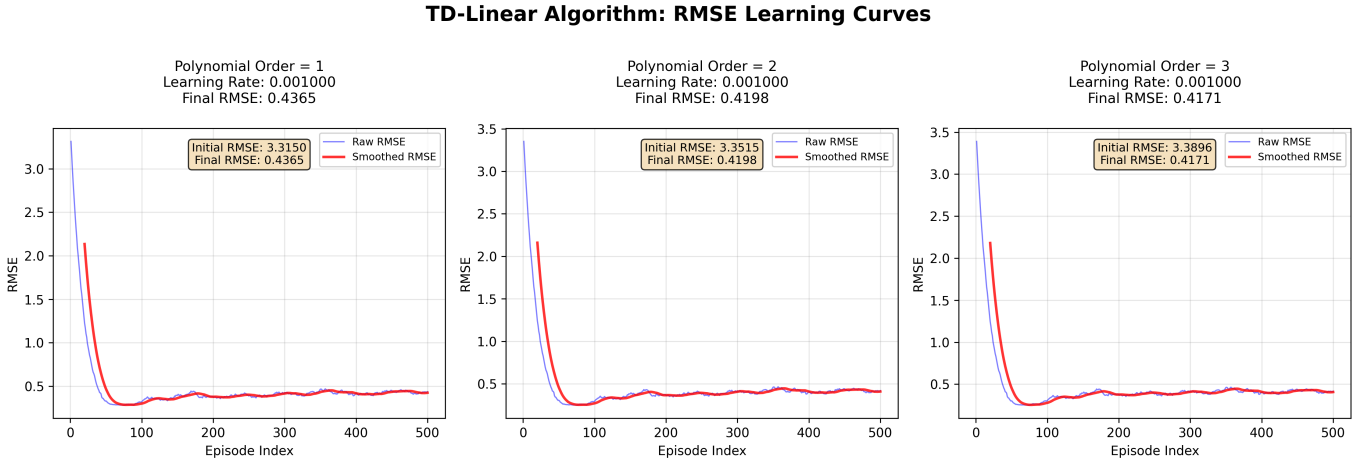
Order	Features
1	`1, x, y`
2	Adds `x^2, xy, y^2`
3	Adds `x^3, x^2y, xy^2, y^3`

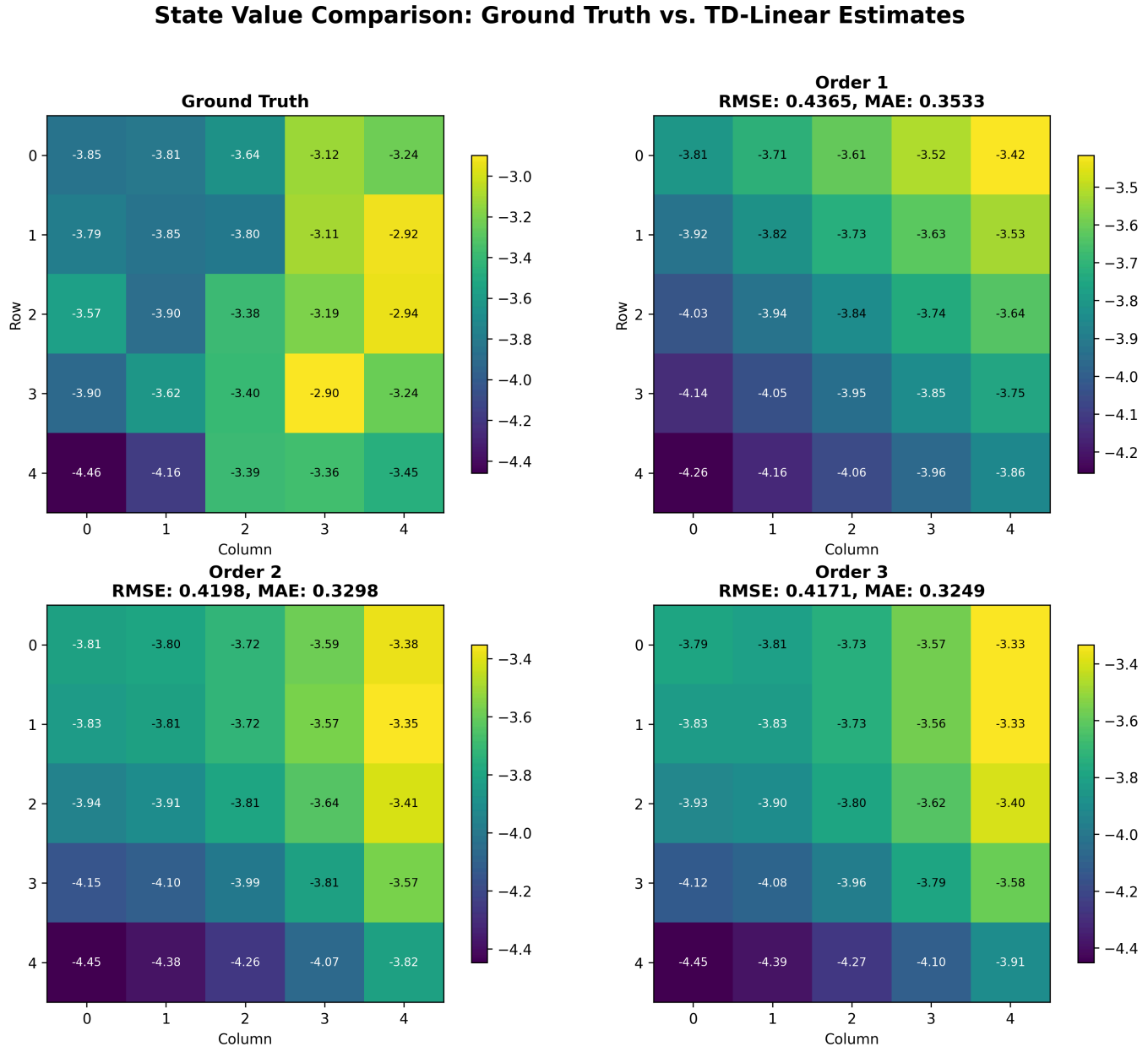
Training uses linear TD(0), learning rate `0.001`, 500 episodes, and 500 steps per episode. The code also normalizes feature norms and clips weight norms for numerical stability.

The README mentions TD(lambda), but the actual experiment implements TD(0). Also, the Bellman reference value is computed under a uniform `0.2` action policy, while the training trajectories use a separately sampled random policy. An additional check shows that these two policy values differ by RMSE about `0.4045`, so not all observed error should be attributed to function approximation.







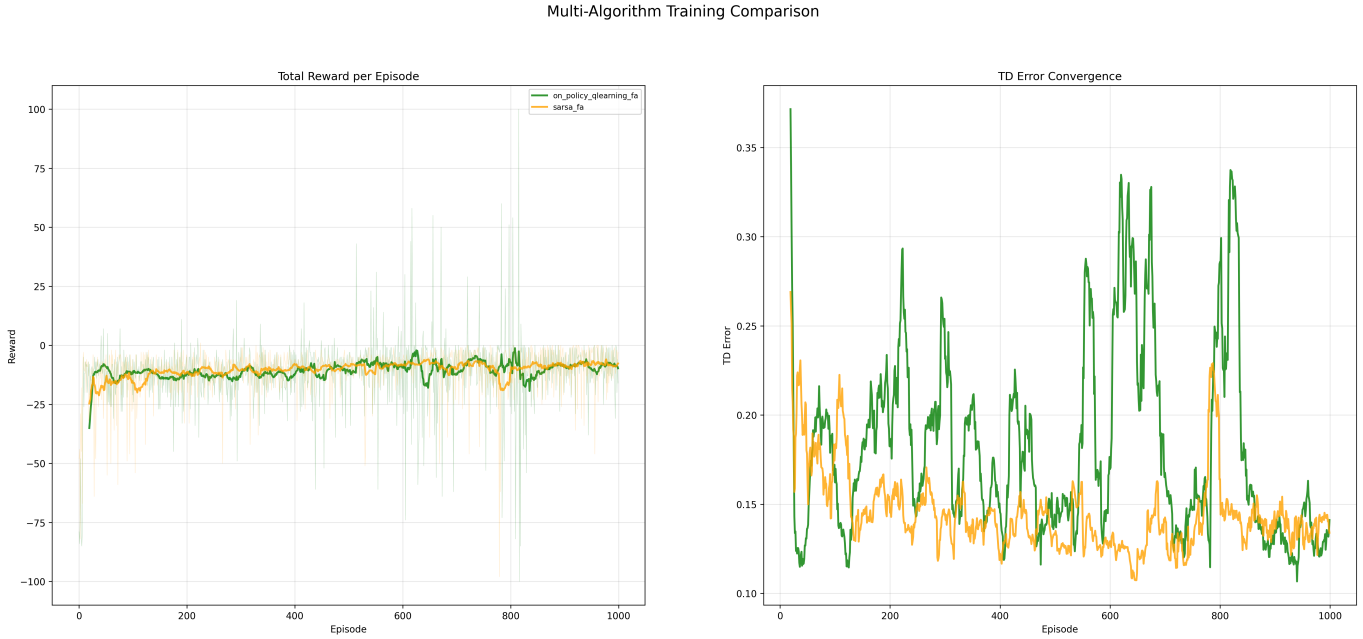


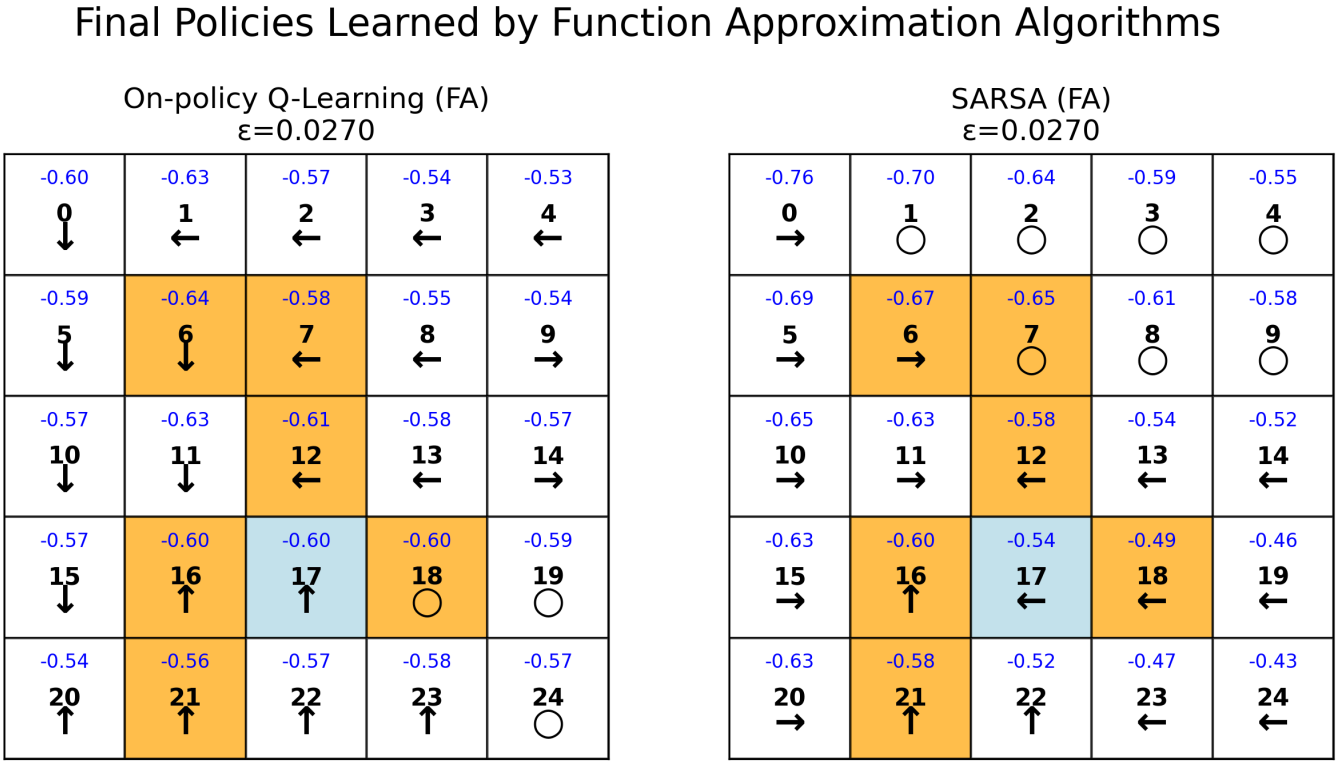
Final RMSE values are close: order 1 around `0.4365`, order 2 around `0.4198`, and order 3 around `0.4171`. Higher-order features help slightly, but the difference is modest.

7.3 Experiment 2: Approximate Control

Experiment 2 applies linear function approximation to SARSA and Q-learning. It uses third-order polynomial features by default and maintains action-specific weights.

Parameter	Value
Episodes	1000
Max steps per episode	100
Learning rate	0.0005
Initial epsilon	0.2
Epsilon decay	0.998
Minimum epsilon	0.01





Under the current hyperparameters, approximate control does not learn as stably as tabular TD. Q-learning has stronger TD-error fluctuations, while SARSA is smoother. This is a useful result because it shows that function approximation adds real optimization and stability issues.

8. Chapter 8: Policy Gradient

8.1 Theory

Value-based methods learn `V` or `Q` first and then derive a policy. Policy-gradient methods directly parameterize:

$$\pi_\theta(a \mid s)$$

The objective is:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$$

The policy-gradient identity is:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(A_t \mid S_t) G_t\right]$$

REINFORCE uses sampled returns:

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)$$

A baseline can reduce variance:

$$\theta \leftarrow \theta + \alpha (G_t - b(S_t)) \nabla_\theta \log \pi_\theta(A_t \mid S_t)$$

Actor-Critic uses a critic to estimate value and an actor to update the policy with an advantage:

$$A_t = G_t - V(S_t)$$

or with TD-style errors:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

Generalized Advantage Estimation combines multi-step TD errors with exponential weighting to trade bias and variance.

8.2 Implementation

Method	Network
REINFORCE	One-hot state input, MLP outputting action probabilities
Actor-Critic	Actor outputs action probabilities, critic outputs state value

In this chapter, the target reward is increased from `1` to `10`. Episodes have maximum length 20, so the undiscounted episode reward upper bound is about `200`.

REINFORCE collects a full trajectory, computes discounted returns, normalizes returns within the episode, and updates the policy. It does not train a separate value baseline network.

Actor-Critic trains actor and critic networks and computes advantages in a GAE-like way. The implementation collects whole trajectories before updating. In the current code, the final time step is not included in the actor update, which slightly affects interpretation but does not change the main idea.

8.2.1 What This Chapter Implements and Why

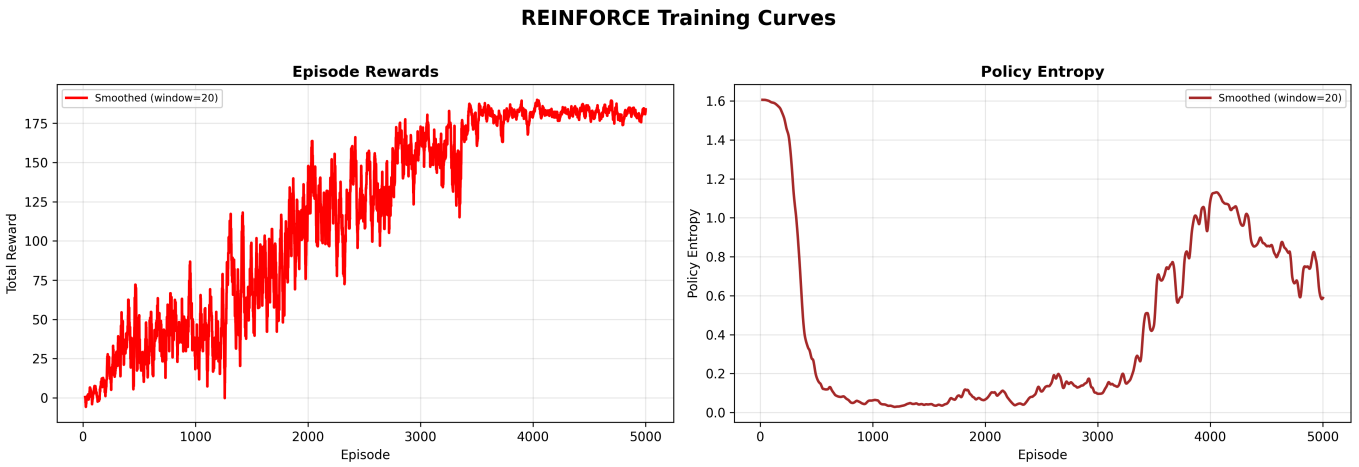
Chapter 8 moves from value-based methods to policy-based methods. This matters when actions are continuous, when the optimal policy should remain stochastic, or when taking `argmax` over all actions is inconvenient.

Method	What It Does	Why It Is Included
REINFORCE	Updates the policy network with full-episode returns	Shows the simplest Monte Carlo policy gradient
Actor-Critic	Trains both policy and value networks	Shows how a critic can reduce gradient variance

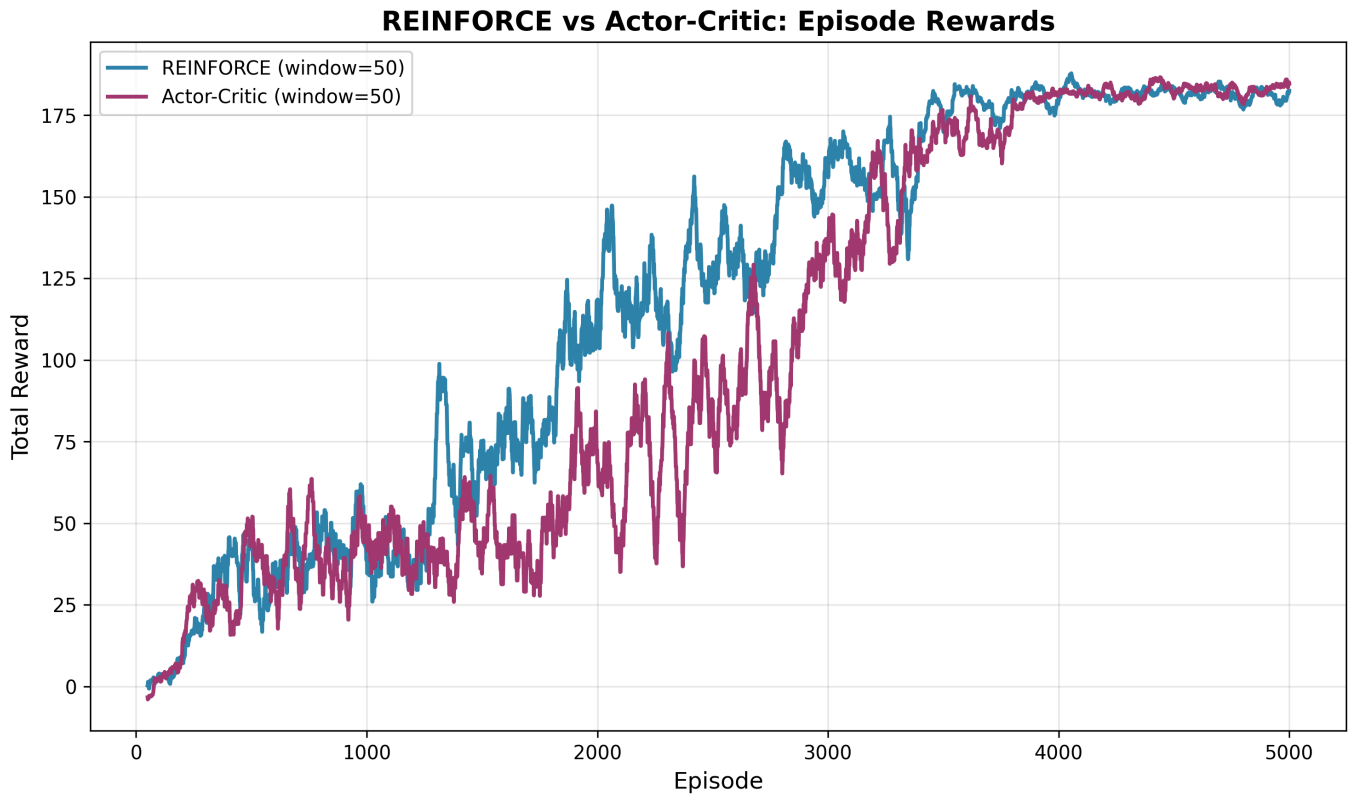
REINFORCE increases the probability of actions followed by high returns and decreases actions followed by low returns. Actor-Critic refines this signal by asking whether an action was better than the state's expected baseline.

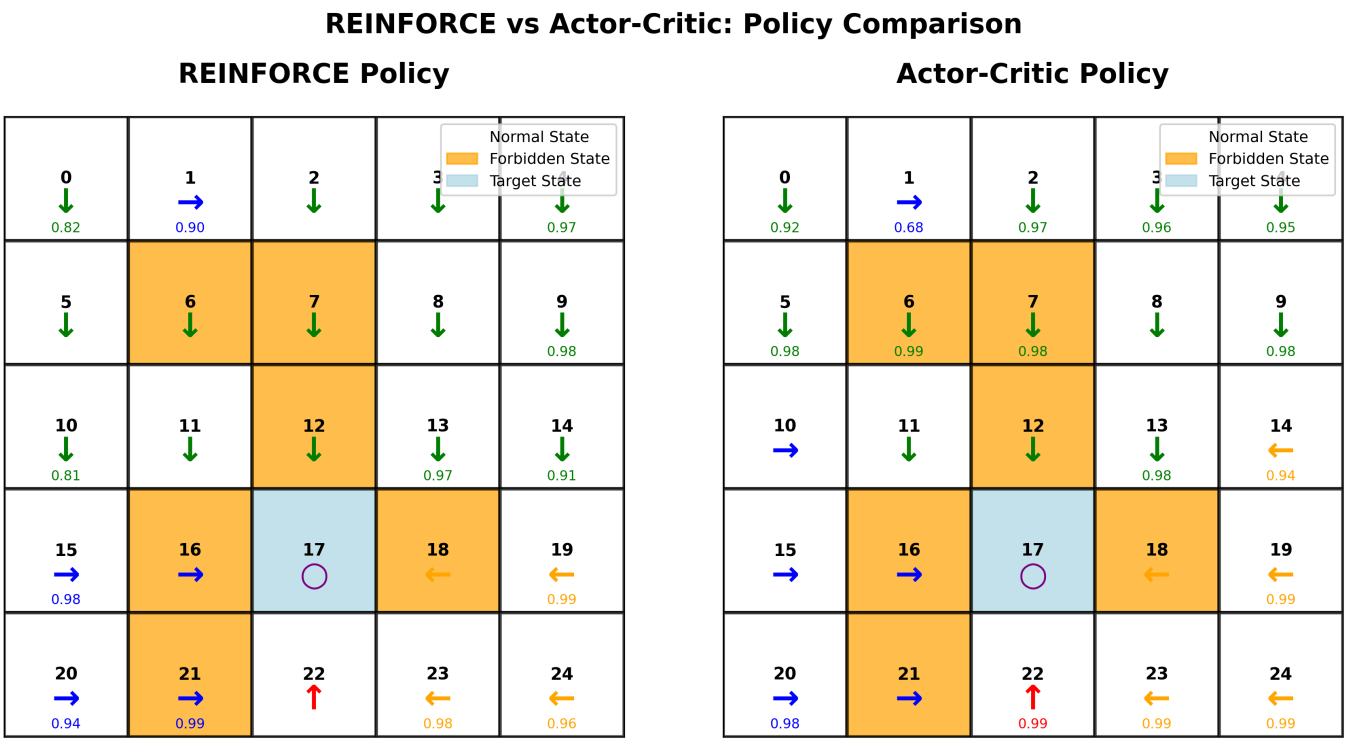
This chapter brings together policies as distributions, values as critics, sampled trajectories as data, stochastic optimization, and TD/GAE-style advantage estimation. Chapter 9 transfers these ideas to token generation and compares PPO's value head with GRPO's group-relative baseline.

8.3 Experimental Results


REINFORCE Training Curves
Episode Rewards
Smoothed (window=20)
Total Reward
Episode
Policy Entropy
Smoothed (window=20)
Policy Entropy
Episode


Actor-Critic Training Curves
Episode Rewards
Raw
Smoothed
Total Reward
Episode
Actor Loss
Actor Loss
Smoothed
Episode
Critic Loss
Critic Loss
Smoothed
Episode


REINFORCE vs Actor-Critic: Episode Rewards
REINFORCE (window=50)
Actor-Critic (window=50)
Total Reward
Episode



In this run, REINFORCE rises earlier in the middle phase, and both methods eventually reach a high-reward region. Because the plot is based on a single random seed, it should not be treated as statistical proof that one method is always better.

9. Chapter 9: From Classical RL to LLM RL with GSM8K, PPO, and GRPO

9.1 How Chapter 8 Leads to Language-Model RL

Chapter 8 already contains the essential pieces of modern policy optimization: an actor outputs action probabilities, a critic estimates state values, sampled trajectories provide returns, advantages compare actions with a baseline, and stochastic gradient descent updates the networks. In an LLM, the same ideas have new concrete forms:

GridWorld / Actor-Critic	GSM8K LLM Training	Connection
State s_t: current grid cell	Question prompt plus generated token prefix	The next action depends on visible context
Action a_t: a grid move	The next token	The policy assigns conditional action probabilities
Policy πθ(a_t | s_t)	Qwen next-token distribution	The actor samples an answer
Trajectory	Complete answer up to EOS	One answer is one episode
Environment reward	Check the final number against the reference answer	Reward defines the optimization target
Critic V(s_t)	PPO value head on hidden states	Estimates expected return from each prefix

GridWorld / Actor-Critic	GSM8K LLM Training	Connection
Advantage A_t	PPO token-level GAE or GRPO group-relative reward	Determines which token probabilities should change

GridWorld can provide feedback after every move. With a GSM8K rule reward, correctness is mainly known only after a complete answer. This creates a credit-assignment problem: how should the final outcome influence earlier tokens? PPO uses a critic and GAE to propagate terminal reward backward. GRPO samples multiple answers to the same question and constructs advantages from their relative rewards. Both extend Chapter 8's advantage-weighted policy gradient.

9.2 Task, Reward, and Data Flow

Each original GSM8K JSONL record has a question and an answer whose final numeric result follows ####. The code puts the question into Qwen's chat template and requests the same final-answer format. src/reward.py extracts the final number, removes thousands separators, and uses Decimal to compare it with the reference. A match receives reward 1; a mismatch receives 0.

```
prompt x  →  sample answer y=(a_1,...,a_T) token by token
          →  extract the final number after ####
          →  rule reward R(x,y) ∈ {0,1}
```

The reward evaluates the final number rather than every reasoning step. Training records update the policy; test records are reserved for final-answer accuracy. Compared with stepwise GridWorld feedback, this terminal reward is sparse, making advantage estimation central to the training flow.

9.3 PPO: One Prompt, One Answer, and a Critic

At each update, src/ppo.py selects one question and samples **one** answer from the current policy. It saves the old policy's token log probabilities and keeps a frozen copy of the initial model as the reference. The reference supplies a KL cost that restrains drift from the starting policy. For valid generated tokens, the code builds:

$$r_t = -\beta \left[\log \pi_{\text{old}}(a_t \mid s_t) - \log \pi_{\text{ref}}(a_t \mid s_t)\right] + \mathbf{1}_{t=T} R(x, y)$$

Here T is the final valid token. Earlier tokens primarily receive the reference-policy KL cost; the terminal token also receives the task reward. A value head reads hidden states and estimates V(s_t), the expected return of each generated prefix. GAE then proceeds backward:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \qquad V(s_{T+1}) = 0$$

$$A_t = \delta_t + \gamma\lambda A_{t+1}, \qquad G_t = A_t + V(s_t)$$

This is a direct continuation of Chapter 6's TD error and Chapter 8's Actor-Critic advantage. PPO compares the current policy with the old policy that sampled the answer, clips their probability ratio to limit large updates, and fits the value target:

$$\rho_t = \exp\left[\log \pi_\theta(a_t \mid s_t) - \log \pi_{\text{old}}(a_t \mid s_t)\right]$$

$$L_{\text{PPO}} = -\mathbb{E}_t\left[\min\left(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t\right)\right] + \frac{c_v}{2}\mathbb{E}_t\left[\left(V_\theta(s_t) - G_t\right)^2\right]$$

The sampled answer can be reused for several PPO epochs. A mask includes the first EOS and earlier tokens, excluding later padding from the loss. PPO thus connects intermediate-state critic values, terminal reward, backward token advantages, and clipped policy updates in one credit-assignment chain.

9.4 GRPO: Compare Answers to the Same Question

src/grpo.py samples a group of answers to one question; the default group size is G=4. Each answer receives an independent 0/1 correctness reward. GRPO does not train a value head. Instead, the group's mean reward becomes a comparison baseline:

$$\bar{R} = \frac{1}{G}\sum_{i=1}^{G} R_i, \qquad A_i = \frac{R_i - \bar{R}}{\text{std}(R_1, \ldots, R_G) + \epsilon}$$

Every valid token in answer i receives the same group-relative advantage A_i. Above-average answers get positive advantages; below-average answers get negative ones. The old-policy probability ratio is clipped, a KL term compares the current and frozen reference policies, and the loss first averages over tokens within an answer and then over answers:

$$\rho_{i,t} = \exp\left[\log \pi_\theta(a_{i,t} \mid s_{i,t}) - \log \pi_{\text{old}}(a_{i,t} \mid s_{i,t})\right]$$

$$L_{\text{GRPO}} = -\mathbb{E}_i\mathbb{E}_t\left[\min\left(\rho_{i,t}A_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)A_i\right)\right] + \beta\mathbb{E}_i\mathbb{E}_t\left[\text{KL}_{i,t}\right]$$

$$\Delta_{i,t} = \log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t}) - \log \pi_\theta(a_{i,t} \mid s_{i,t}), \qquad \text{KL}_{i,t} = \exp(\Delta_{i,t}) - \Delta_{i,t} - 1$$

This replaces Chapter 8's learned baseline with a comparison among answers to the same question. If every answer receives the same reward, all group-relative advantages are zero. The policy signal therefore depends on differences in answer quality, not merely the number of samples.

9.5 Implementation, Defaults, and What to Observe

File / Module	Responsibility	Reason for the Separation
scripts/download_gsm8k.py and src/data.py	Download and read original train/test JSONL	Keep data handling separate from training
src/reward.py	Parse the number after #### and assign binary reward	Share one reward rule across PPO, GRPO, and evaluation
src/modeling.py	Chat template, sampling, EOS mask, token log probabilities, saving	Give both algorithms the same model interface
src/ppo.py	One-answer rollout, value head, GAE, clipped update	Extend Chapter 8 Actor-Critic to token-level credit assignment
src/grpo.py	Grouped rollout, normalized relative advantage, clipped update	Show advantage estimation without training a critic

File / Module	Responsibility	Reason for the Separation
src/evaluate.py	Greedy test generation and exact final-answer accuracy	Use the same answer criterion for training and evaluation

The default model is Qwen/Qwen2.5-0.5B-Instruct. Both training entry points use up to 64 training questions, 20 updates, and at most 128 new tokens by default. PPO samples one answer per question; GRPO samples four. The chapter requirements list only PyTorch and Transformers, and the data download uses the standard library. From the repository root:

```
python Chapter9_LLM_RL_GSM8K/scripts/download_gsm8k.py
python -m Chapter9_LLM_RL_GSM8K.src.ppo --steps 20
python -m Chapter9_LLM_RL_GSM8K.src.grpo --steps 20 --group-size 4
python -m Chapter9_LLM_RL_GSM8K.src.evaluate --model
Chapter9_LLM_RL_GSM8K/outputs/grpo
```

For PPO, inspect per-question reward, policy loss, and value loss together. For GRPO, inspect mean group reward and within-group reward standard deviation. Both methods sample data under the current policy and then use reward-derived advantages to change it. Their key difference is that PPO learns an intermediate-state value function while GRPO uses relative comparisons among answers to the same question.

10. Relationship Across Chapters

The first eight chapters place classical RL algorithms in a shared GridWorld; Chapter 9 moves to GSM8K to show how those ideas transfer. The relationship can be summarized by chapter and implementation:

Chapter	Implemented Algorithm/Module	What It Does	Why It Is Included	Key Experimental Notes
Chapter 1	GridWorld environment	Builds states, actions, forbidden cells, target cell, reward matrix `R`, and transition tensor `P`	Provides the shared MDP for the GridWorld experiments in Chapters 1–8	Target is nonterminal; `stay` can repeatedly collect reward
Chapter 1	Random policy and value visualization	Generates a policy matrix and plots policy/value	Makes policy and value concrete	Random policy values are not concentrated around the target
Chapter 2	Closed-form Bellman evaluation	Solves $(I - \gamma P_\pi)v = r_\pi$	Shows exact policy evaluation in small finite MDPs	More expensive for large problems
Chapter 2	Iterative Bellman evaluation	Repeatedly applies Bellman backups	Shows value propagation and convergence	Difference from closed form is about `9.2e-6`

Chapter	Implemented Algorithm/Module	What It Does	Why It Is Included	Key Experimental Notes
Chapter 2	Policy comparison	Compares value functions under several policies	Shows that policy choice changes long-term return	Random policy is normalized random, not exactly uniform
Chapter 3	Policy Iteration	Fully evaluates, then greedily improves a policy	Shows the classic control loop	Clean theory, heavier evaluation
Chapter 3	Value Iteration	Applies optimal Bellman backups	Solves optimal control more directly	Final value nearly matches PI
Chapter 3	Truncated Policy Iteration	Performs partial evaluation before improvement	Shows computation-accuracy tradeoff	Larger `x` approaches full PI
Chapter 4	Basic Monte Carlo	Estimates `Q(s,a)` from fixed short trajectories	Introduces model-free return estimation	Short trajectories introduce truncation bias
Chapter 4	Exploring Starts MC	Samples random initial `(s,a)`	Improves state-action coverage	Implementation is closer to last-visit after backward deduplication
Chapter 4	Epsilon-Greedy MC	Explores inside trajectories	More practical model-free control	Exploration rate shapes the visit distribution
Chapter 5	BGD	Uses all samples per update	Low-noise optimization baseline	Smooth trajectory
Chapter 5	SGD	Uses one random sample per update	Shows noisy online optimization	Stronger oscillation
Chapter 5	MBGD	Uses a mini-batch per update	Balances noise and computation	Explains later RL curve fluctuations
Chapter 6	SARSA	Uses the sampled next action in TD target	On-policy TD control	Learns values under exploratory behavior
Chapter 6	Expected SARSA	Uses expected next value under the policy	Reduces next-action sampling variance	TD error drops quickly in this run

Chapter	Implemented Algorithm/Module	What It Does	Why It Is Included	Key Experimental Notes
Chapter 6	n-step SARSA	Uses multi-step returns before bootstrapping	Connects MC and TD(0)	Larger `n` changes variance and target scale
Chapter 6	Online Q-learning	Updates step by step with greedy target	Learns toward optimal action values	Conceptually off-policy despite naming
Chapter 6	Offline-style Q-learning	Collects a trajectory then updates	Compares update workflow	Not modern dataset-only offline RL
Chapter 7	Linear TD(0) prediction	Approximates `V(s)` with polynomial features	Moves beyond tabular values	Actual code is TD(0), not TD(lambda)
Chapter 7	Feature-order comparison	Compares 1st, 2nd, and 3rd order features	Studies representation capacity	Reference and sampling policies differ
Chapter 7	Approximate SARSA	Uses linear `Q(s,a;w)` for control	Tests on-policy approximate control	Rewards improve only modestly
Chapter 7	Approximate Q-learning	Uses linear `Q(s,a;w)` for control	Tests off-policy approximate control	TD error fluctuates strongly
Chapter 8	REINFORCE	Updates policy from full returns	Basic Monte Carlo policy gradient	Return normalization, no learned value baseline
Chapter 8	Actor-Critic	Trains actor and critic networks	Introduces advantage-based policy learning	GAE-like advantage; final step not used in actor update
Chapter 8	Policy-gradient comparison	Compares reward curves and final policies	Summarizes value-based to policy-based transition	Single-seed curves are not statistical conclusions
Chapter 9	GSM8K rule reward	Extracts the number after #### and assigns 0/1 reward	Turns answer correctness into a policy-optimization signal	Final-answer-only reward is sparse
Chapter 9	PPO	Samples one answer and updates with a value head, GAE, and clipping	Extends Chapter 8 Actor-Critic to token-level credit assignment	Inspect policy loss, value loss, and reference-policy KL

Chapter	Implemented Algorithm/Module	What It Does	Why It Is Included	Key Experimental Notes
Chapter 9	GRPO	Samples multiple answers to one question and uses normalized group rewards	Replaces a trained critic with relative comparisons	Equal rewards within a group produce zero relative advantage
Chapter 9	Test evaluation	Generates greedily and measures final-answer accuracy	Applies one answer rule to saved policies	Loads only the saved policy

The learning path is:

```
MDP modeling
  -> Bellman equations
  -> Dynamic programming
  -> Monte Carlo learning
  -> Temporal-difference learning
  -> Function approximation
  -> Policy gradients
  -> LLM RL: answers as trajectories and tokens as actions
  -> PPO / GRPO: value-based or group-relative advantages for policy updates
```

The first steps establish classical RL; Chapter 9 shows how the same ideas enter language-model reasoning training.

11. Reproduction Notes

Install the repository dependencies with:

```
pip install -r requirements.txt
```

Each chapter has `scripts/*.sh`. They can be run on Linux, macOS, WSL, or Git Bash. On Windows PowerShell, the shell scripts should be translated into the corresponding Python commands.

Suggested reproduction order:

1. Run Chapters 1 and 2 to verify GridWorld, policy matrices, and value visualization.
2. Run Chapter 3 to compare PI, VI, and TPI.
3. Run Chapters 4 and 6 to compare MC and TD without using the model.
4. Run Chapter 5 separately to build intuition for learning rates and noisy optimization.
5. Run Chapters 7 and 8 to study function approximation, policy gradients, and critics.
6. Then move to Chapter 9: read PPO's one-answer/value-head path before GRPO's grouped comparisons, download GSM8K, and use the training entry points documented there.

Possible improvements for a more rigorous report:

Improvement	Reason
Run multiple random seeds	Single curves do not prove statistical significance
Save raw reward/error logs	Enables means, variances, and confidence intervals
Unify episode length and reward scale	Makes cross-chapter comparisons cleaner
Explicitly choose terminal or continuing target behavior	Episodic and continuing tasks have different value interpretations
Align Chapter 7 training and reference policies	Avoid attributing policy mismatch to function approximation error
Fix Chapter 8 final-step advantage handling	Makes actor updates cover the full trajectory
Compare Chapter 9 group sizes and reward designs	Shows how GRPO's relative advantages and policy signal change

12. Summary

This project implements foundational RL algorithms from scratch. The first eight chapters use GridWorld to move from known-model planning to model-free learning, from tabular values to function approximation, and from value learning to direct policy optimization. Chapter 9 moves to GSM8K and shows how those concepts support language-model training. Chapters 1 to 3 establish MDPs, Bellman equations, and dynamic programming; Chapters 4 to 6 learn from sampled experience through Monte Carlo and TD; Chapter 7 adds function approximation; Chapter 8 implements REINFORCE and Actor-Critic; and Chapter 9 treats answers as trajectories and tokens as actions, using PPO's critic/GAE or GRPO's within-group comparisons to construct advantages.

The experimental results broadly match RL theory: dynamic programming is stable in small known-model settings; Monte Carlo methods depend on exploration and trajectory length; TD methods use bootstrapping; and function approximation and policy gradients depend more strongly on representation, sampling, and optimization choices. PPO and GRPO then show how the same policy-gradient framework can handle terminal rewards in language-model training through two different advantage estimators. The project now connects observable classical RL experiments to LLM generation and policy optimization.

Because both the code and this document are built for learning and experimentation, there may be implementation details, formula explanations, or experimental interpretations that can be improved. If you find issues in the code, formulas, plots, or explanations, corrections through issues, pull requests, or direct feedback are welcome.

13. References

- Project repository: https://github.com/tequila28/Foundations-of-RL-Learning
- Mathematical Foundations of Reinforcement Learning: https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning

- Sutton, R. S. and Barto, A. G. Reinforcement Learning: An Introduction, 2nd edition. http://incompleteideas.net/book/the-book-2nd.html
- Schulman et al. High-Dimensional Continuous Control Using Generalized Advantage Estimation. https://arxiv.org/abs/1506.02438
- Schulman et al. Proximal Policy Optimization Algorithms. https://arxiv.org/abs/1707.06347
- Shao et al. DeepSeekMath (introducing GRPO). https://arxiv.org/abs/2402.03300
- Original GSM8K data and format: https://github.com/openai/grade-school-math
- Qwen2.5-0.5B-Instruct model card: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct